



Picodata

Руководство администратора

Содержание

Обзор методов конфигурирования	11
Типы параметров	11
Постоянные	11
Изменяемые	11
Использование файла конфигурации	11
Использование переменных окружения	12
Использование параметров командной строки	12
Приоритеты методов конфигурирования	13
Получение данных о кластере	14
Получение лидера raft-группы	14
Получение состава raft-группы	14
Получение версии схемы данных	15
Просмотр отладочного журнала	15
Журнал raft-лидера	16
Получение данных по HTTP API	16
Параметры инстанса и кластера	17
Оценка состояния инстанса для Kubernetes	18
Метрики инстанса	18
Включение сбора метрик	19
Метрики в консоли	19
Dashboard для Grafana	20
Возможности и требования	20
Настройка для развернутого кластера	20
Настройка источника данных	20
Настройка Grafana	21
Развертывание кластера через Ansible	23

Требования к оборудованию и ПО	23
Установка роли	23
Создание директорий	23
Создание инвентарного файла	23
Простой кластер на нескольких серверах	24
Кластер из двух тиров на нескольких серверах	24
Изменение параметров роли	25
Расчет количества инстансов/репликасетов	25
Количество репликасетов в кластере	25
Количество инстансов на одном сервере	26
Создание плейбука	26
Установка кластера	26
Управление кластером	26
Установка пакета picodata	27
Получение информации о кластере	27
Удаление кластера	27
Добавление новых инстансов в кластер на существующих серверах	27
Ребутстрап инстанса	27
Резервное копирование и восстановление кластера	28
Управление плагинами	28
Действия при авариях	29
Мониторинг кластера	29
Развёртывание через Kubernetes Operator	30
Требования	30
Установка оператора	30
Установка через Helm	30
Установка оператора	30

Развёртывание кластера через Helm	31
Обновление кластера	31
Удаление	31
Создание namespace и секрета	31
Описание ресурса PicoclusterDB	32
Развёртывание кластера	32
Минимальный кластер	32
Наблюдение за запуском	33
Фазы кластера	33
Подключение к кластеру	34
Протокол PostgreSQL (psql)	34
Веб-интерфейс (Web UI)	34
Внешний доступ	34
Ingress	34
LoadBalancer / NodePort	35
Per-pod-сервисы для клиентов, работающих с шардированными данными	35
Настройка MetalLB для per-pod VIP	36
Shard-aware без LoadBalancer: DNS + BGP pod routing	38
Управление плагинами	39
Сборка образа с плагином	39
Описание плагина в манифесте	39
Обновление версии Picodata	40
Ребутстрап инстанса	40
Резервное копирование	41
Настройка хранилища резервных копий	41
Ограничения резервного копирования	41
Создание резервной копии	42
Ограничения восстановления	42
Восстановление из резервной копии	42

Anti-affinity	43
Требование к StorageClass	43
Отключение anti-affinity	44
PodDisruptionBudget	44
Удаление кластера	44
Мониторинг	44
ServiceMonitor через CR	45
PodMonitor с job-лейблом по имени кластера	45
Grafana-дашборд	45
Настройка серверов для кластера	46
Эксплуатация и мониторинг	46
Настройка службы времени	46
Настройка DNS	46
Настройка журналирования	46
Настройка SELinux	46
Настройка мониторинга	47
Настройка доступности портов	47
Настройка сохранения дампов памяти при сбоях	47
Настройка лимитов для ресурсов	47
Настройка Journald	48
Безопасность	48
Настройка пользователей	48
Настройка аудита	48
Производительность	48
Настройка дисковой подсистемы	48
Настройка swap	49
Установка Picodata	49
Выбор версии	49
Выбор топологии	49

Предоставление данных	49
Управление кластером в промышленной среде с ограниченными привилегиями.	
Общие сведения	50
Требования для установки кластера	50
Настройки для Systemd	51
Первоначальная настройка серверов	51
Инсталляция кластера	51
Управление кластером без повышения привилегий	51
Удаление кластера	52
Очистка серверов от системных файлов кластера	52
Настройки для Supervisord	52
Работа с кластером через Supervisord	52
Обновление кластера	54
Обновление с остановкой кластера	54
Отключение плагинов	54
Остановка инстансов	54
Обновление ПО	55
Запуск после обновления	55
Обновление и включение плагинов	55
Обновление без остановки кластера	56
Действия после обновления кластера	56
Проверка гетерогенности кластера	56
Проверка применения нового набора системных таблиц	56
Удаление инстансов из кластера вручную	57
Откат обновления	57
Тестирование производительности	58
Системные требования	58
Использование стандартного теста TPC-B	58

Предварительные действия	58
Установка, инициализация и запуск теста	58
Изменение параметров теста	58
Использование собственных тестов	59
Резервное копирование и восстановление	60
Общие сведения	60
Копирование всего кластера	60
С помощью SQL	60
С помощью Ansible	60
Копирование отдельных инстансов	61
Создание локальной резервной копии	61
Восстановление из локальной резервной копии	61
Управление доступом	62
Ролевая модель	62
Объекты доступа	63
Пользователи СУБД	63
Привилегии	64
Роли	66
Начало работы	66
Управление пользователями	66
Методы аутентификации	66
Создание	67
Модификация	67
Блокирование	68
Удаление	68
Просмотр списка пользователей	68
Установка и изменение пароля	68
Требования к паролю	68

Использование ролей	69
Управление доступом к веб-интерфейсу	69
Управление доступом к таблицам	70
Управление доступом к процедурам	70
Аутентификация с помощью LDAP	71
Общие сведения	71
Механизм аутентификации пользователей	71
Особенности работы LDAP	71
Поддержка защищённого режима	71
Настройка	72
Настройка инстанса для аутентификации LDAP	72
Настройка пользователей LDAP в Picodata	73
Устранение неполадок	74
Подключение к кластеру в Oracle Weblogic	75
Общие сведения	75
Предварительные требования	75
Подготовка окружения	75
Настройка источников данных	76
Добавление источника	76
Объединение источников	80
Безопасность кластера	83
Подключение по протоколу PostgreSQL в безопасном режиме	83
Настройка на стороне сервера	83
Настройка на стороне клиента	84
Аутентификация по сертификату	84
Создание сертификатов и ключей	85
Настройки для mTLS-коммуникации внутри кластера	85
Настройка mTLS для коммуникации внутри кластера	86

Настройка HTTPS для безопасного доступа к метрикам и веб-интерфейсу	87
Использование журнала аудита	88
Общие сведения	88
Способы ведения журнала	88
Включение журнала	88
В командной строке	88
В файле конфигурации	89
С помощью переменной окружения	89
С помощью Ansible	89
Получение доступа к журналу	90
Включение журнала для DML-операций	90
Оповещения о событиях безопасности	90
Поддерживаемые способы доставки оповещений	90
Оповещения для рабочего стола	91
Оповещения по электронной почте	91
Рекомендации по сайзингу	93
Задачи сайзинга	93
Распределение имеющихся ресурсов	93
Вычисление нагрузки	93
Расчет ожидаемого размера БД	93
Расчет индексов БД	95
Расчет требуемого объема для движка memtx	95
Пример расчета размера таблицы	95
Настройка Systemd	97
Настройка лимитов в инвентарном файле	97
Системные ограничения	97
Проверка лимитов	97

Устранение неполадок	99
Не удастся включить плагин	99
Пользователь не может авторизоваться	99
Невозможно подключиться к инстансу	100
Выпадение инстанса из кластера	100
Невозможность добавления новых инстансов в кластер	101
Сообщение об ошибке повторного подключения к репликасету	102
Информационная безопасность	103
Действия по приемке поставленного ПО	103
Проверка требований к сырью, материалам и комплектующим	103
Проверка комплектности	103
Проверка комплектности и качества эксплуатационной документации	103
Проверка маркировки и упаковки	103
Действия по проверке контрольных сумм	104
Действия по безопасной установке и настройке изделия	104
Действия по реализации функций безопасности среды функционирования изделия	104

Обзор методов конфигурирования

В данном разделе описаны способы и особенности конфигурирования инстансов Picodata.

Типы параметров

Постоянные

Постоянные параметры задаются один раз при запуске инстанса и затем не могут быть изменены.

Постоянные параметры, относящиеся к глобальным настройкам кластера, устанавливаются в момент первоначальной сборки кластера и не могут быть изменены без его повторной инициализации (bootstrap). Например:

- PICODATA_CLUSTER_NAME (имя кластера)

Постоянные параметры, относящиеся к отдельным инстансам, также задаются один раз в начале при запуске инстанса. Менять их впоследствии нельзя. Например:

- PICODATA_INSTANCE_TIER (имя тира, которому будет принадлежать инстанс)
- PICODATA_INSTANCE_NAME (имя инстанса)

Изменяемые

Изменяемые параметры могут быть переопределены для действующих инстансов в кластере. Такие параметры, в свою очередь, делятся на:

- изменяемые без перезапуска инстанса
- изменяемые через перезапуск инстанса

Примеры изменяемых параметров:

- SQL_VDBE_OPCODE_MAX / SQL_MOTION_ROW_MAX (неблокирующие запросы)
- PICODATA_CONFIG_FILE (путь к файлу конфигурации)
- PICODATA_CONFIG_PARAMETERS (список пар ключ-значение)

Для изменения параметров, не требующих перезапуска инстанса, можно использовать SQL-команду ALTER SYSTEM.

Остальные параметры можно изменить, указав их новые значения при повторном запуске инстанса (см. способы задания параметров ниже).

Использование файла конфигурации

Параметры запуска инстанса можно указать в файле конфигурации, и затем передать путь к нему в качестве аргумента для `picodata run`.

Данный способ удобен для:

- запуска нескольких инстансов и работы с кластерными возможностями Picodata
- разделения глобальных настроек кластера и настроек отдельных инстансов

- удобного контроля над большим списком переопределяемых параметров запуска каждого инстанса
- использования разных наборов настроек для инстансов исходя из их назначения (тиры, домены отказа и т.д.)

Файл конфигурации представляет собой текстовый файл с YAML-разметкой.

Пример:

my_cluster.yml

```
cluster:
  name: my_cluster
instance:
  instance_dir:
    ./mini1
  ipproto:
    listen:
      127.0.0.1:3308
```

Примечание

Файл конфигурации должен содержать имя кластера

Запуск инстанса с использованием файла конфигурации:

```
picodata run --config my_cluster.yml
```

Использование переменных окружения

У всех параметров запуска Picodata имеются аналогичные переменные окружения, которые можно задавать в командной оболочке (например, Bash).

В примере выше был показан запуск инстанса с явным указанием его рабочей директории и сетевого адреса. При использовании переменных окружения это будет выглядеть так:

```
export PICODATA_INSTANCE_DIR="./data/i1"
export PICODATA_IPROTO_LISTEN="127.0.0.1:3301"
picodata run
```

Данный способ удобен для запуска нескольких инстансов (с разными параметрами) на одном хосте. Наборы команд можно сохранить в виде shell-скриптов, индивидуальных для каждого инстанса.

Использование параметров командной строки

Исполняемый файл `picodata`, запускающий инстанс Picodata, поддерживает разнообразные дополнительные параметры, с помощью которых можно явно указать рабочую директорию инстанса, сетевой порт и т.д.

Данный способ удобен для:

- быстрого ознакомления с Picodata в UNIX shell на примере 1 или пары инстансов
- наглядного и оперативного контроля над всеми параметрами запуска инстанса Picodata

Пример использования аргументов командной строки:

```
picodata run --instance-dir ./data/i1 --iproto-listen 127.0.0.1:3301
```

Приоритеты методов конфигурирования

Поскольку разные методы установки параметров можно использовать одновременно, важно учитывать их приоритет. В Picodata установлены следующие приоритеты (от высшего к низшему):

1. Аргумент командной строки
2. Переменная окружения
3. Значение в файле конфигурации

Пример:

```
cat my_cluster.yaml
cluster:
  name: my_cluster
instance:
  instance_dir:
    ./mini1
  ipproto_listen:
    127.0.0.1:3302
```

```
export PICODATA_IPROTO_LISTEN="127.0.0.1:3303"
picodata run --iproto-listen 127.0.0.1:3304 --config my_cluster.yaml
```

После запуска сетевой адрес инстанса будет 127.0.0.1:3304.

Получение версии схемы данных

Узнать текущую версию схему данных можно с помощью команды:

```
box.space._pico_property:get("current_schema_version");
---
- ['current_schema_version', 1]
...
```

Каждое изменение схемы данных в кластере приводит к увеличению этого номера.

Просмотр отладочного журнала

Способ просмотра отладочного журнала инстансов зависит от того, как был запущен кластер Picodata.

Если при запуске инстанса командой `picodata run` в консоли не был явно указан способ вывода отладочного журнала, то он выводится в ту же консоль.

Если кластер развернут при помощи файла конфигурации, то в нем вывод отладочного журнала может быть настроен в файл или в службу `syslog`. Этим поведением управляет параметр конфигурации `instance.log.destination`.

При развёртывании кластера через Ansible отладочный журнал по умолчанию выводится в службу `syslog`.

Для доступа к отладочному журналу:

- при выводе в файл используйте команду `tail -f` или любой другой инструмент просмотра файлов
- при выводе журнала в `syslog` используйте команду `journalctl`

При использовании службы `syslog` для просмотра журнала конкретного инстанса вам понадобится знать имя его сервиса. При развёртывании через Ansible оно по умолчанию формируется как `<cluster_name>@<instance_name>.service`. Например, для кластера с именем `test` и инстанса `default-1000` команда для просмотра отладочного журнала будет такой:

```
journalctl -u test@default-1000.service
```

Пример вывода:

```
systemd[1]: Starting Picodata cluster test@default-1000...
systemd[1]: Started Picodata cluster test@default-1000.
picodata[4731]: 'cluster.name': "test"
picodata[4731]: 'cluster.tier': {"default": {"replication_factor": 3, "bucket_count": 3000, "can_vote": true}}
picodata[4731]: 'cluster.default_replication_factor': 1
picodata[4731]: 'instance.instance_dir': "/var/lib/picodata/test/default-1000"
picodata[4731]: 'instance.name': "default-1000"
picodata[4731]: 'instance.ipproto.listen': "0.0.0.0:13301"
picodata[4731]: 'instance.http.listen': "0.0.0.0:18001"
picodata[4731]: 'instance.admin_socket': "/var/run/picodata/test/default-1000.sock"
picodata[4731]: [supervisor:4731] running StartDiscover
picodata[4733]: entering discovery phase
...
```

Журнал raft-лидера

Отладочный журнал raft-лидера представляет особый интерес при диагностике неполадок в кластере т.к. содержит важную информацию об изменениях, происходящих с конфигурацией и топологией. Raft-лидером в каждый момент времени является только один инстанс в кластере и именно он управляет остальными. Для доступа к отладочному журналу raft-лидера выполните следующие шаги.

Вычислите raft-лидера в кластере. Для этого подключитесь к административной консоли любого инстанса:

```
picodata admin ./admin.sock
```

Переключите язык ввода на Lua и узнайте идентификатор текущего raft-лидера:

```
\lua;
pico.raft_status();
```

Пример вывода:

```
---
- main_loop_status: idle
  leader_id: 2
  id: 1
  term: 30
  raft_state: Follower
...
```

С помощью полученного `leader_id` выясните адрес сервера, на котором запущен этот инстанс:

```
\sql;
SELECT "name", "_pico_instance"."raft_id", "_pico_peer_address"."address"
FROM "_pico_instance" JOIN "_pico_peer_address"
ON "_pico_peer_address"."raft_id"="_pico_instance"."raft_id"
WHERE "_pico_instance"."raft_id" = 2;
```

Пример вывода:

```
+-----+-----+-----+
| name           | raft_id | address           |
+=====+=====+=====+
| "default-2000" | 2       | "192.168.0.2:3301" |
+-----+-----+-----+
(1 rows)
```

Дальнейшие шаги зависят от варианта запуска. При развёртывании кластера через Ansible потребуется подключиться к серверу по SSH и открыть диагностический журнал через `journalctl`, см. Просмотр отладочного журнала.

Получение данных по HTTP API

Инстанс Picodata отдаёт сведения о своём состоянии и работоспособности по HTTP API на ряд эндпоинтов, описанных ниже. HTTP-код 200 означает положительный результат, 503 — ошибку.

Для формирования корректного URL используйте адрес, указанный для параметра `HTTP_LISTEN`, например:

```
http://127.0.0.1:8081/api/v1/health/status
```

Параметры инстанса и кластера

Эндпоинт `/api/v1/health/status` требует авторизации на инстансе Picodata. Для получения данных авторизации отправьте в терминале запрос следующего вида, указав данные ранее созданного пользователя:

```
curl -X POST http://127.0.0.1:8081/api/v1/session -H "Content-Type: application/json" -d '{
  "username": "alice",
  "password": "T0psecret"
}' | jq
```

Примечание

В этом и последующих примерах используется форматирование JSON-вывода при помощи консольной утилиты `jq`.

В ответ Picodata пришлёт два токена (`auth` и `refresh`). Пример:

```
{
  "auth": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGJjZSI6ImV4cCI6MTc3MzQ5NzQwNSwidHlwIjoieYXV0aCI6InJvbGVzIjpbImV4ZWV1dGU1LCJjcmVhdGUiLCJyZWFiIiwid3JpdGU1LCJsb2dpbiIsImFsdGVyIiI19.-NX0ef7e9ckRwGLJBuz0dXtRhI2nBhEZj5P1Wip7w30",
  "refresh": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGJjZSI6ImV4cCI6MTc3MzQ5NzQwNSwidHlwIjoicmVmcVzaCj9.gEP-qLjhlIhgzg_05-8ntAaf9MeH4qiFw0ib_DiRM7U"
}
```

Токен `auth` действует в течение 24 часов. Теперь вы можете получить данные о состоянии инстанса, указав токен в заголовке `Authorization: Bearer <token>`:

```
curl -s "http://127.0.0.1:8081/api/v1/health/status" -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGJjZSI6ImV4cCI6MTc3MzQ5NzQwNSwidHlwIjoieYXV0aCI6InJvbGVzIjpbImV4ZWV1dGU1LCJjcmVhdGUiLCJyZWFiIiwid3JpdGU1LCJsb2dpbiIsImFsdGVyIiI19.-NX0ef7e9ckRwGLJBuz0dXtRhI2nBhEZj5P1Wip7w30" | jq
```

Пример вывода

```
{
  "status": "healthy",
  "timestamp": 1773411090,
  "uptimeSeconds": 14573,
  "name": "i1",
  "uuid": "2c4a425b-49c7-4e0f-86bd-1a479cd6ce4a",
  "version": "25.5.0-1848-g41efc83dc",
  "raftId": 1,
  "tier": "default",
  "replicaset": "default_1",
  "currentState": "Online",
  "targetState": "Online",
  "targetStateReason": "wakeUp",
  "targetStateChangeTime": "2026-03-12 14:45:55.637123218 +00:00:00",
  "limboOwner": 0,
  "raft": {
    "state": "Follower",
    "term": 9,
    "leaderId": 3,
    "leaderName": "i3",
    "appliedIndex": 162,
    "committedIndex": 162,
    "compactedIndex": 0,
    "persistedIndex": 162
  },
  "buckets": {
    "active": 8192,
    "pinned": 0,
    "sending": 0,
    "receiving": 0,
    "garbage": 0,
    "total": 16384
  },
  "cluster": {
    "uuid": "24502c90-de10-4172-9a4c-82b8856caaf1",
    "version": "25.5.0-1609-g92f7550fa"
  }
}
```

Токен `refresh` действует в течение 1 года. Он предназначен для того, чтобы обновить `auth`-токен без предоставления данных авторизации. Пример запроса на выдачу новых токенов с помощью `refresh`-токена:

```
curl -s "http://127.0.0.1:8081/api/v1/session" -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhbGJjZSIsImV4cCI6MTgwNDk0NzAwNSwidHlwIjoicmVmcmVzaCJ9.gEP-qLjJhIhZg_05-8ntAaf9MeH4qiFw0ib_DiRM7U" | jq
```

Оценка состояния инстанса для Kubernetes

Эндпоинты, предназначенные для оценки состояния инстанса в Kubernetes, не требуют авторизации и могут использоваться как в адресной строке веб-браузера, так и в терминале (например, с `curl`). Доступны следующие эндпоинты:

Эндпоинт	Описание	HTTP-коды	Авторизация
/api/v1/health/startup	Признак того, что инстанс успешно загрузился	200/503	не требуется
/api/v1/health/live	Признак того, что инстанс доступен в сети	всегда 200	не требуется
/api/v1/health/ready	Признак того, что инстанс готов к работе	200/503	не требуется

Примечание

По умолчанию, данные эндпоинты в Picodata включены и доступны без дополнительной настройки. В файле конфигурации можно явным образом отключить их, указав параметр `instance.http.kubernetes_probes` со значением `false`.

Оценка успешного запуска инстанса (`startup`) строится на следующих критериях:

- состояние инстанса оценивается как `Online`
- Raft-лидер в кластере определён
- репликaset, в котором состоит инстанс, наполнен (в таблице `_pico_replicaset` в столбце `state` для него должно быть указано `ready`).

Для успешного ответа при обращении к эндпоинту `/ready` достаточно первых двух критериев.

Согласно спецификации Kubernetes, эндпоинты `/ready` и `/live` будут отдавать валидное состояние только после того, как `/startup` вернул код 200. Иными словами, нет смысла запрашивать `/ready` или `/live` до того как инстанс успешно запустится и `/startup` вернёт код 200.

Пример положительного ответа на запрос состояния:

```
{"status": "ok"}
```

Пример отрицательного ответа:

```
Failure (503):  
{"status": "not_ready", "reason": "instance state is Offline"}
```

Метрики инстанса

Функциональность сбора метрик позволяет получать параметры работы СУБД из инстанса Picodata и предоставлять доступ к ним для внешних систем в формате Prometheus.

Включение сбора метрик

Модуль сбора метрик автоматически включается при использовании встроенного HTTP-сервера в Picodata. Для этого нужно при запуске инстанса использовать параметр `--http-listen` и задать адрес веб-сервера. Например:

```
picodata run --http-listen '127.0.0.1:8081'
```

Метрики в консоли

Метрики инстанса Picodata можно получить в консоли, используя `curl`:

```
curl --location 'http://127.0.0.1:8081/metrics'
```

Dashboard для Grafana

В данном разделе приведена информация об установке dashboard для мониторинга запущенного кластера Picodata в Grafana. Dashboard представляет собой интерактивную приборную панель, которая отображает данные в реальном времени и позволяет в удобном и наглядном виде визуализировать важные параметры кластера, следить за его производительностью, оперативно реагировать на инциденты и устранять неполадки.

Возможности и требования

Мы предоставляем заранее сконфигурированный dashboard-файл Picodata.json, который можно импортировать в интерфейсе Grafana и использовать для отслеживания состояния кластера, изменения состава и роли узлов, а также для оперативного доступа к метрикам Picodata.

Для использования dashboard-файла необходимо:

- иметь доступ к развернутому кластеру Picodata
- настроить источник данных для Grafana

Настройка для развернутого кластера

Настройка источника данных

Перед настройкой Grafana сконфигурируйте и запустите систему мониторинга Prometheus, которая является источником данных для Grafana.

Установите и настройте Prometheus согласно рекомендациям для вашей ОС. Укажите для Prometheus цели в файле /etc/prometheus/prometheus.yml:

```
Пример минимальной конфигурации global:
  scrape_interval: 10s

scrape_configs:
- job_name: 'picodata'
  scrape_interval: 5s
  metrics_path: /metrics
  static_configs:
    - targets: ['127.0.0.1:8081']
```

В приведенном примере:

- 127.0.0.1:8081 — адрес, с которого Prometheus собирает метрики (должен соответствовать параметру --http-listen при запуске инстанса Picodata или параметру instance.http.listen при использовании файла конфигурации)

Под каждый инстанс Picodata нужно выделять отдельный адрес сбора метрик. Например, если локально запустить 4 инстанса Picodata, то файл конфигурации Prometheus может выглядеть так:

```
global:
  scrape_interval: 10s

scrape_configs:
- job_name: 'picodata'
  scrape_interval: 5s
  metrics_path: /metrics
```

```
static_configs:
  - targets: ['127.0.0.1:8081', '127.0.0.1:8082', '127.0.0.1:8083', '127.0.0.1:8084']
```

В промышленной эксплуатации кластера для получения списка значений для поля `targets` используйте роль `Ansible`: после выполнения плейбука будет создан `yaml`-файл `report.yml`, в котором будет, в том числе, готовая секция `scrape_configs` с перечислением всех инстансов и портов кластера.

Настройка Grafana

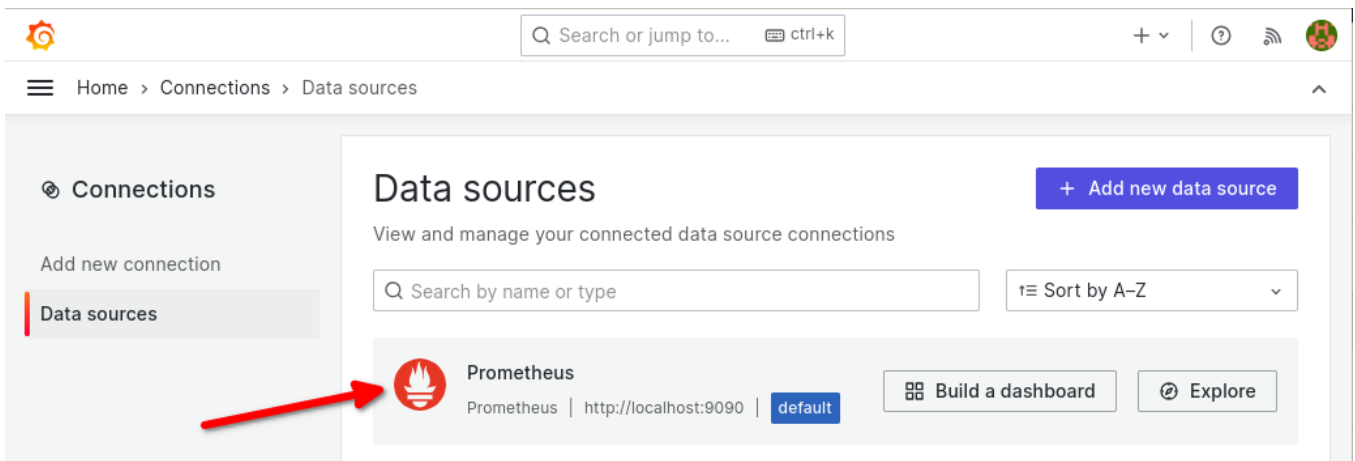
После того как Prometheus сконфигурирован и запущен, установите и настройте Grafana согласно рекомендациям для вашей ОС. Удостоверьтесь, что Grafana работает, открыв в веб-браузере URL, на котором она запущена.

Примечание

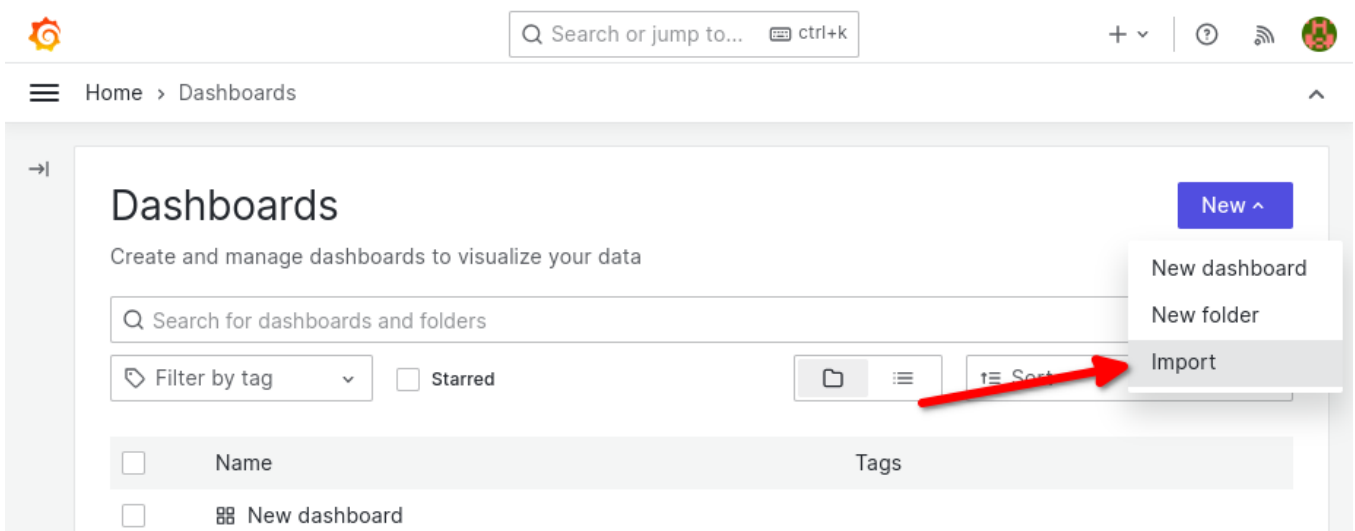
URL для доступа к Grafana задается в файле `grafana.ini`. По умолчанию это `127.0.0.1:3000`.

Далее выполните следующие шаги.

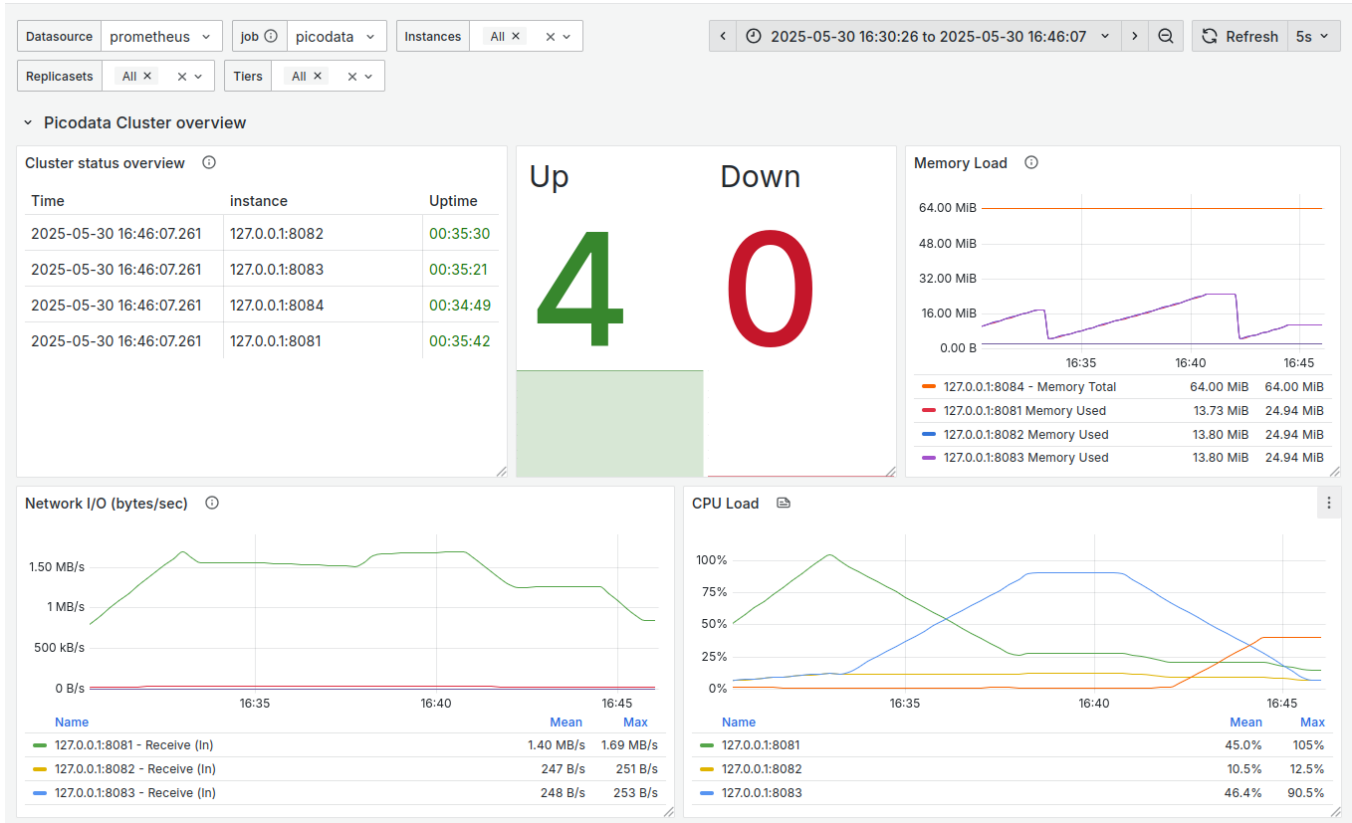
1. Убедитесь, что в настройках подключений в Grafana (`Connections > Data sources`) имеется источник данных Prometheus:



2. Импортируйте dashboard с данными Picodata. Для этого понадобится файл `Picodata.json`, который следует добавить в меню `Dashboards > New > Import`:



После этого в Grafana можно будет оперативно отслеживать состояние инстансов, потребляемую память, нагрузку на сеть, изменения в составе кластера и прочие параметры:



Развертывание кластера через Ansible

В данном разделе приведена информация по развёртыванию кластера Picodata из нескольких экземпляров, запущенных на разных серверах посредством роли `picodata-ansible`.

Внимание!

Информация приведена для версии роли 25.5.1 и выше

Требования к оборудованию и ПО

Для промышленной эксплуатации кластера следует учитывать базовые требования к оборудованию и программному обеспечению:

- наличие подготовленных серверов с поддерживаемыми ОС на базе Linux (см. список)
- минимальный расчет ресурсов на 1 экземпляр (без учета ресурсов для ОС): 1 ядро ЦП, 64 МБ ОЗУ, 512 МБ дискового пространства
- у пользователя, запускающего роль, должны быть права на запись в текущей директории
- версия `ansible`: 2.9 – 2.15
- версия `python`: 3.8 – 3.13

Установка роли

Установите роль из репозитория через `ansible-galaxy`:

```
ansible-galaxy role install -f picodata.picodata_ansible
```

или создайте файл `requirements.yml` со следующим содержимым:

```
roles:
  - name: picodata.picodata_ansible
```

При необходимости укажите нужную версию роли: `roles: - name: picodata.picodata_ansible version: 26.2.1`

и затем выполните команду:

```
ansible-galaxy install -fr requirements.yml
```

Создание директорий

В текущей директории создайте поддиректории — это нужно для отделения инвентарных файлов от сценариев (плейбуков):

```
mkdir {hosts,playbooks}
```

Создание инвентарного файла

Рассмотрим два сценария: простой кластер с единственным типом и кластер из нескольких типов. Пусть в распоряжении есть два сервера: `192.168.0.1` и `192.168.0.2`.

Создайте инвентарный файл `hosts/cluster.yml` и наполните его в зависимости от нужного сценария содержанием ниже.

Простой кластер на нескольких серверах

cluster.yml

```
all:
  vars:
    install_packages: true          # для установки пакета picodata из репозитория
    cluster_name: simple_cluster   # имя кластера
    first_bin_port: 13301          # начальный бинарный порт для первого инстанса
    first_http_port: 18001         # начальный http-порт для первого инстанса для веб-интерфейса

  tiers:
    default:
      instances_per_server: 2      # описание тиров
                                   # имя тира (default)
                                   # сколько инстансов запустить на каждом сервере этого тира

G1:
  hosts:
    server-1:
      ansible_host: 192.168.0.1    # имя датацентра (используется для failure_domain)
                                   # далее перечисляем серверы в датацентре
                                   # имя сервера в инвентарном файле (используется для failure_domain)
                                   # IP-адрес или fqdn если не совпадает с предыдущей строкой
G2:
  hosts:
    server-2:
      ansible_host: 192.168.0.2    # имя датацентра (используется для failure_domain)
                                   # далее перечисляем серверы в датацентре
                                   # имя сервера в инвентарном файле (используется для failure_domain)
                                   # IP-адрес или fqdn если не совпадает с предыдущей строкой
```

Кластер из двух тиров на нескольких серверах

Пример инвентарного файла для 4-х серверов, расположенных в 3-х датацентрах (DC1, DC2 и DC3):

cluster.yml

```
all:
  vars:
    ansible_user: vagrant          # пользователь для ssh-доступа к серверам

    repo: 'https://download.picodata.io' # репозиторий, откуда устанавливать пакет picodata

    cluster_name: 'demo'           # имя кластера
    admin_password: '123asdZXV'    # пароль пользователя admin

    default_bucket_count: 23100    # количество бакетов в каждом тире (по умолчанию 30000)

    audit: false                   # состояние аудита (отключен)
    log_level: 'info'              # уровень отладки
    log_to: 'file'                 # вывод журнала в файлы (а не в journald)

    conf_dir: '/etc/picodata'       # директория для хранения конфигурационных файлов
    data_dir: '/var/lib/picodata'   # директория для хранения данных
    run_dir: '/var/run/picodata'    # директория для хранения sock-файлов
    log_dir: '/var/log/picodata'    # директория для журналов и файлов аудита
    share_dir: '/usr/share/picodata' # директория для размещения служебных данных (плагинов)

    listen_address: '{{ ansible_fqdn }}' # адрес, который будет слушать инстанс. Для IP указать {{ ansible_default_ipv4.address }}
    pg_address: '{{ listen_address }}'   # адрес, который будет слушать PostgreSQL-протокол инстанса

    first_bin_port: 13301           # начальный бинарный порт для первого инстанса
    first_http_port: 18001          # начальный http-порт для первого инстанса для веб-интерфейса
    first_pg_port: 15001            # начальный номер порта для PostgreSQL-протокола инстансов кластера

  tiers:
    arbiter:
      replicaset_count: 1          # описание тиров
      replication_factor: 1        # имя тира
      config:                      # количество репликасетов
        memtx:                     # фактор репликации
          memory: 64M              # количество памяти, выделяемое каждому инстансу тира
      host_groups:
        - ARBITERS                # целевая группа серверов для установки инстансов тира

    default:
      replicaset_count: 3          # имя тира
      replication_factor: 3        # количество репликасетов
      bucket_count: 16384          # фактор репликации
      config:                      # количество бакетов в тире
        memtx:                     # количество памяти, выделяемое каждому инстансу тира
          memory: 71M              # количество памяти, выделяемое каждому инстансу тира
      host_groups:
        - STORAGES                # целевая группа серверов для установки инстансов тира

  db_config:                       # параметры конфигурации кластера (см. https://docs.picodata.io/picodata/stable/reference/db_config)
```

```

governor_auto_offline_timeout: 30
iproto_net_msg_max: 500
memtx_checkpoint_count: 1
memtx_checkpoint_interval: 7200

plugins:
example:
  path: '../plugins/weather_0.1.0-ubuntu-focal.tar.gz' # плагин
  config: '../plugins/weather-config.yml' # путь до пакета плагина
  # путь до файла с настройками плагина
  services:
    weather_service:
      tiers:
        - default # список тиров, в которые устанавливается сервис плагина
                  # указано значение по умолчанию (default)

DC1:
hosts: # имя датацентра (failure_domain)
  server-1-1: # серверы в датацентре
    ansible_host: '192.168.19.21' # имя сервера в инвентарном файле
    host_group: 'STORAGES' # IP-адрес или fqdn если не совпадает с предыдущей строкой
                        # определение целевой группы серверов для установки инстансов

  server-1-2: # имя сервера в инвентарном файле
    ansible_host: '192.168.19.22' # IP-адрес или fqdn если не совпадает с предыдущей строкой
    host_group: 'ARBITERS' # определение целевой группы серверов для установки инстансов тира arbiter

DC2:
hosts: # имя датацентра (failure_domain)
  server-2-1: # серверы в датацентре
    ansible_host: '192.168.20.21' # имя сервера в инвентарном файле
    host_group: 'STORAGES' # IP-адрес или fqdn если не совпадает с предыдущей строкой
                        # определение целевой группы серверов для установки инстансов

DC3:
hosts: # имя датацентра (failure_domain)
  server-3-1: # серверы в датацентре
    ansible_host: '192.168.21.21' # имя сервера в инвентарном файле
    host_group: 'STORAGES' # IP-адрес или fqdn если не совпадает с предыдущей строкой
                        # определение целевой группы серверов для установки инстансов

```

Изменение параметров роли

Параметры роли, используемые по умолчанию, располагаются в файле `defaults/main.yml`; их также можно переопределять в инвентарном файле.

Помимо параметров по умолчанию, используется словарь `tiers` с указанием внутри:

- имени тира (например `arbiter`)
- количества инстансов на каждом сервере (`instances_per_server`) или количества репликасетов (`replicaset_count`)
- фактора репликации (`replication_factor`) для тира (по умолчанию равен 1)

Пример словаря `tiers`:

```

tiers:
  default:
    replicaset_count: 2 # имя тира default
    replication_factor: 3 # количество репликасетов
    bucket_count: 16384 # фактор репликации
    config:
      memtx:
        memory: 128M # количество бакетов в тире
                  # количество памяти, предоставляемое непосредственно на хранение данных

```

Расчет количества инстансов/репликасетов

В инвентарном файле для тиров можно указывать количество как инстансов на каждом сервере, так и репликасетов. При этом, во втором случае значение `instances_per_server` будет рассчитано автоматически. Если при расчете будет получено дробное значение, то роль остановится с ошибкой.

Если указаны оба параметра, то приоритет у `instances_per_server`

Количество репликасетов в кластере

```
replicaset_count = instances_per_server * SERVER_COUNT / replication_factor
```

Если при расчете вы получаете дробное число, значит количество серверов или фактор репликации подобраны некорректно и это необходимо исправить.

Количество инстансов на одном сервере

```
instances_per_server = replicaset_count * replication_factor / SERVER_COUNT
```

Аналогично, если при расчете вы получаете дробное число, значит количество серверов или фактор репликации подобраны некорректно и это необходимо исправить.

Создание плейбука

Для подключения роли создайте плейбук `playbooks/picodata.yml`, добавив в него следующее содержание:

`picodata.yml`

```
---
- name: Deploy picodata cluster
  hosts: all
  become: true

  tasks:

    - name: Import picodata-ansible role
      ansible.builtin.import_role:
        name: picodata-ansible
```

Установка кластера

Выполните установку кластера командой:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml
```

При успешном окончании выполнения плейбука будет создан `yaml`-файл `report.yml` с перечислением всех инстансов и портов кластера.

Более подробно о доступных переменных в инвентарном файле можно узнать в статье [Переменные, используемые в роли Ansible](#)

Управление кластером

Для управления кластером используйте теги роли, указав их после параметра `-t`.

Список поддерживаемых тегов:

Имя	Описание
backup	Создание резервной копии кластера
command	Выполнение <code>lua</code> -команд на инстансах
crash_dump	Сбор файлов необходимых для анализа разработчиками в случае сбоя кластера
deploy	Установка кластера. Используется по умолчанию, если не указаны другие теги
deploy_become, remove_become	Используются для установки/удаления кластера в режиме <code>rootless</code>
expand	Добавление новых инстансов в кластер на существующих серверах

Имя	Описание
genin	Получить список инстансов в разбивке для каждого сервера (используется в других сценариях)
install_pkgs	Установка пакета Picodata. Также используется при установке кластера
plugins	Установка и конфигурация плагинов
rebootstrap	Ребутстрап инстанса
remove	Удаление кластера
report	Генерация файлов report
restart	Перезапуск всех инстансов на всех серверах
restore	Восстановление из резервной копии кластера
restore_full	Установка кластера и восстановление из резервной копии
start	Запуск всех инстансов на всех серверах после остановки
stop	Остановка всех инстансов на всех серверах

Установка пакета picodata

Пример команды для установки пакета picodata (если не указана переменная `picodata_package_path`, то на серверах будет подключен репозиторий `picodata`):

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t install_pkgs
```

Получение информации о кластере

Пример команды для получения информации об именах инстансов кластера с их привязкой к серверам:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t genin
```

Удаление кластера

Пример команды для удаления кластера:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t remove
```

Добавление новых инстансов в кластер на существующих серверах

Данная процедура применяется только для добавления новых инстансов на существующие серверы.

Внимание!

Для добавления новых серверов в кластер нужно запускать роль с тегом `deploy`

Укажите новое количество инстансов или репликasetов в инвентарном файле и выполните команду:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t expand
```

Ребутстрап инстанса

Применяется в случае выхода инстанса из строя и невозможности его ввода в репликaset кластера в обычном порядке.

Внимание!

Проводить ребутстрап инстанса можно только в репликасетах с фактором репликации 2 и более, при этом инстанс не должен быть последним работающим в репликасете

Пример команды для ребутстрапа инстанса с именем `default_1_2`:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t rebootstrap -e "instance=default_1_2"
```

Резервное копирование и восстановление кластера

Пример команды для создания резервной копии кластера:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t backup
```

Пример команды для восстановления кластера в случае, если кластер уже развернут:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t restore
```

Пример команды для восстановления кластера на пустых серверах (будет развернут кластер и выполнено восстановление из последней резервной копии):

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t restore_full
```

Пример команды для восстановления кластера из определенной резервной копии:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t restore -e restore_dir=20260916203059
```

Внимание!

Восстановление из последней резервной копии работает только при выставленной переменной `backup_fetch` в `true` (т.е. когда резервные копии выкачиваются на станцию запуска Ansible). В остальных случаях необходимо указывать переменную `restore_dir`!

Управление плагинами

С помощью роли `picodata-ansible` можно также добавлять в кластер плагины и их конфигурации. Для этого модифицируйте инвентарный файл `hosts/cluster.yml`, добавив в него блок `plugins`. Пример инвентарного файла, поддерживающего установку в кластер с одним тиром тестового плагина *weather*:

`cluster.yml`

```
all:
  vars:
    install_packages: true                # для установки пакета picodata из репозитория
    cluster_name: simple_cluster         # имя кластера
    first_bin_port: 13301                 # начальный бинарный порт для первого инстанса (он же main_peer)
    first_http_port: 18001                # начальный http-порт для первого инстанса для веб-интерфейса

  tiers:
    default:
      bucket_count: 20000                 # описание тиров
      instances_per_server: 2              # имя тира (default)
                                          # сколько бакетов будет размещено на данном тире
                                          # сколько инстансов запустить на каждом сервере этого тира

  plugins:
    example:
      path: '../plugins/weather_0.1.0.tar.gz' # описание плагинов
      config: '../plugins/weather-config.yml'  # имя плагина в Ansible (может не совпадать с именем в Picodata)
      services:
        weather_service:
          tiers:
            - default                      # путь к архиву с плагином
                                          # путь к файлу конфигурации плагина
                                          # список тиров, в которые устанавливается сервис плагина
                                          # указано значение по умолчанию (default)
```

```

G1:                                     # имя группы серверов (используется для failure_domain)
  hosts:                             # далее перечисляем серверы в датацентре
    server-1:                         # имя сервера в инвентарном файле (используется для failure_domain)
      ansible_host: 192.168.0.1      # IP-адрес или fqdn если не совпадает с предыдущей строкой
G2:                                     # имя группы серверов (используется для failure_domain)
  hosts:                             # далее перечисляем серверы в датацентре
    server-2:                         # имя сервера в инвентарном файле (используется для failure_domain)
      ansible_host: 192.168.0.2      # IP-адрес или fqdn если не совпадает с предыдущей строкой

```

Для подключения плагина потребуется сформировать архив из его файлов, а также подготовить файл конфигурации плагина.

Модифицировать файл плейбука не требуется.

Выполните установку плагинов в кластер командой:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t plugins
```

Примечание

С помощью роли Ansible можно только добавлять и обновлять плагины в кластере. Удаление производится вручную.

Действия при авариях

Пример команды для сбора информации после сбоя кластера для отправки ее в техподдержку Picodata:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml -t crash_dump
```

Будет собрана вся информация для расследования инцидента. После выполнения следует передать файлы "`*_crash_dump.tar.gz`" из директории `backup_fetch_dir/cluster_name/` (`backup_fetch_dir` и `cluster_name` — переменные из инвентарного файла. Если не определены, то используются значения по умолчанию) в техническую поддержку компании Picodata.

Мониторинг кластера

Для мониторинга развернутого кластера Picodata используйте dashboard для Grafana. Мы предоставляем заранее сконфигурированный dashboard-файл Picodata.json, который можно импортировать в интерфейсе Grafana и использовать для отслеживания состояния кластера (в том числе, отслеживать изменения состава и роли узлов), а также для оперативного доступа к метрикам Picodata.

Dashboard позволяет следить за производительностью кластера, оперативно реагировать на инциденты и устранять неполадки.

Для удобства добавления инстансов в конфигурационный файл Prometheus можно взять секцию `prometheus` из файла `report.yml`, который создается после выполнения роли Ansible.

Развёртывание через Kubernetes Operator

Kubernetes Operator — рекомендуемый способ управления кластером Picodata в Kubernetes для production-окружений. Оператор реализует reconcile-loop на основе собственного набора объектов (CRD, Custom Resource Definition) PicoclusterDB и автоматически поддерживает желаемое состояние кластера, создавая и обновляя контроллеры StatefulSet, сервисы (Services), хранилища конфигураций (ConfigMaps), а также следит за установкой и обновлением плагинов.

Требования

- Kubernetes 1.30+
- kubectl 1.28+

Установка оператора

Установка выполняется в два шага: сначала регистрируются CRD, затем разворачивается сам оператор.

Шаг 1. Установите CRD:

```
kubectl apply -f https://git.picodata.io/core/picodata-operator/-/raw/stable/picodata-operator-deploy/crd.yaml
```

Шаг 2. Установите оператор:

```
kubectl apply -f https://git.picodata.io/core/picodata-operator/-/raw/stable/picodata-operator-deploy/operator.yaml
```

Дождитесь готовности пода оператора:

```
kubectl rollout status deployment/picodata-operator-controller-manager \
-n picodata-operator-system
```

Установка через Helm

Helm-чарты позволяют управлять оператором и кластерами декларативно, без ручного применения манифестов.

Установка оператора

Установите оператор с помощью Helm:

```
helm install picodata-operator \
https://git.picodata.io/core/picodata-operator/-/archive/stable/chart.tar.gz \
-n picodata-operator-system --create-namespace
```

Дождитесь готовности:

```
kubectl rollout status deployment/picodata-operator-controller-manager \
-n picodata-operator-system
```

Обновление CRD

Helm добавляет CRD при первой установке, но **не обновляет** их при `helm upgrade`. При обновлении версии оператора применяйте CRD отдельно:

```
kubectl apply -f https://git.picodata.io/core/picodata-operator/-/raw/vX.Y.Z/picodata-operator-deploy/crd.yaml
```

Развёртывание кластера через Helm

Создайте файл значений `values.yaml` с конфигурацией кластера:

```
values.yaml:
image:
  tag: "26.1.5"

adminPassword:
  password: "<пароль>"

cluster:
  defaultReplicationFactor: 2
  defaultBucketCount: 3000

tiers:
- name: default
  replicaset: 2
  replicationFactor: 2
  storage:
    size: 10Gi
  memtx:
    memory: "512M"
  pg:
    enabled: true
  resources:
    requests:
      cpu: "500m"
      memory: "512Mi"
```

Установите кластер:

```
helm install my-cluster \
  https://git.picodata.io/core/picodata-operator/-/archive/stable/chart-cluster.tar.gz \
  -n picodata --create-namespace \
  -f values.yaml
```

Следите за готовностью:

```
kubectl get picoclusterdb -n picodata -w
```

Обновление кластера

Для обновления версии Picodata или изменения конфигурации:

```
helm upgrade my-cluster ./chart-cluster -n picodata --set image.tag=26.2.0
```

Удаление

`helm uninstall` **не удаляет** ресурс PicoclusterDB — данные защищены от случайного удаления. Для полного удаления кластера:

```
kubectl delete picoclusterdb -n picodata my-cluster-picodata-cluster
helm uninstall my-cluster -n picodata
```

PVC (данные) остаются после удаления CR. Для полного удаления данных:

```
kubectl delete pvc -n picodata -l app.kubernetes.io/instance=my-cluster
```

Создание namespace и секрета

Создайте namespace для кластера и секрет с паролем пользователя admin:

```
kubectl create namespace picodata

kubectl create secret generic picodata-admin-secret \
  --namespace picodata \
  --from-literal=password=<ваш-пароль>
```

Внимание!

Пароль должен соответствовать политике безопасности Picodata: не менее 8 символов, латинские буквы в верхнем и нижнем регистрах, цифры.

Описание ресурса PicoclusterDB

Кластер описывается ресурсом PicoclusterDB. Основные поля:

```
apiVersion: picodata.picodata.io/v1
kind: PicoclusterDB
metadata:
  name: <имя-кластера>
  namespace: <namespace>
spec:
  image:
    repository: docker.binary.picodata.io # реестр образов
    tag: "picodata:26.1.5" # образ и версия Picodata
    pullPolicy: IfNotPresent

  clusterName: <имя-кластера> # имя кластера внутри Picodata

  adminPassword:
    secretName: picodata-admin-secret # Secret с паролем admin
    key: password

  cluster:
    defaultReplicationFactor: 1 # фактор репликации по умолчанию для всех тиров
    defaultBucketCount: 3000 # количество бакетов (шардов)

  tiers: # список тиров кластера
    - name: <имя-тира>
      replicaset: 1 # количество репликасетов в тире
      replicationFactor: 1 # количество инстансов в каждом репликасете
      canVote: true # участвует ли тир в Raft-голосовании
      storage:
        size: 1Gi
      memtx:
        memory: "128M"
      pg:
        enabled: true # включить протокол PostgreSQL
      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
```

Общее количество подов тира равно `replicaset × replicationFactor`.

Имена подов строятся по шаблону:

{тир}-{кластер}-{номер-репликасета}-{номер-инстанса}.

Развёртывание кластера**Минимальный кластер**

Кластер из двух тиров: арбитр для Raft и основной тир для хранения данных. Пример минимальной конфигурации:

```
picodata-cluster.yamlapiVersion: picodata.picodata.io/v1
kind: PicoclusterDB
metadata:
  name: picodata
  namespace: picodata
spec:
  image:
    repository: docker.binary.picodata.io
    tag: "picodata:26.1.5"
```

```

pullPolicy: IfNotPresent

clusterName: picodata

adminPassword:
  secretName: picodata-admin-secret
  key: password

cluster:
  defaultReplicationFactor: 1
  defaultBucketCount: 3000

tiers:
  - name: arbiter
    replicaset: 1
    replicationFactor: 1
    canVote: true
    storage:
      size: 1Gi
    memtx:
      memory: "64M"
    pg:
      enabled: false
    resources:
      requests:
        cpu: "100m"
        memory: "128Mi"

  - name: default
    replicaset: 1
    replicationFactor: 2
    canVote: false
    storage:
      size: 10Gi
    memtx:
      memory: "512M"
    pg:
      enabled: true
    resources:
      requests:
        cpu: "500m"
        memory: "512Mi"

```

Примените манифест:

```
kubectl apply -f picodata-cluster.yaml
```

Наблюдение за запуском

Следите за готовностью подов:

```
kubectl get pods -n picodata -w
```

Ожидаемый результат (все поды 1/1 Running):

NAME	READY	STATUS	RESTARTS	AGE
arbiter-picodata-1-0	1/1	Running	0	3m
default-picodata-1-0	1/1	Running	0	3m
default-picodata-1-1	1/1	Running	0	2m

Проверьте состояние CR:

```
kubectl get picoclusterdb -n picodata
```

NAME	READY	AGE
picodata	true	5m

Фазы кластера

Поле `.status.phase` отражает текущее состояние кластера:

Фаза	Описание
Pending	пользовательский ресурс (CR) создан, оператор начинает первичную инициализацию
Initializing	Ресурсы (StatefulSet, Service) создаются, кластер формируется
Ready	Все поды запущены, кластер готов к работе
Degraded	Часть подов недоступна, кластер работает в деградированном режиме
Unknown	Состояние кластера не удалось определить
Deleting	CR помечен на удаление, идёт очистка ресурсов

Наблюдайте за сменой фаз в реальном времени:

```
kubectl get picoclusterdb -n picodata -w
```

При удалении (`kubectl delete picoclusterdb`) оператор переводит кластер в фазу `Deleting` и удерживает CR через финализатор до завершения очистки. После этого CR исчезает из API.

Подключение к кластеру

Протокол PostgreSQL (psql)

Пробросьте pgproto-порт одного из подов типа `default`:

```
kubectl port-forward -n picodata pod/default-picodata-1-0 5432:5432
```

Подключитесь через `psql`:

```
psql "host=localhost port=5432 user=admin password=<пароль> dbname=picodata sslmode=disable"
```

Проверьте состояние инстансов через системную таблицу `_pico_instance`:

```
SELECT name, replicaset_name, current_state, tier
FROM _pico_instance;
```

Веб-интерфейс (Web UI)

Пробросьте HTTP-порт:

```
kubectl port-forward -n picodata svc/default-picodata 8081:8081
```

Откройте `http://localhost:8081` для просмотра состояния кластера в веб-интерфейсе.

Внешний доступ

Ingress

Ingress обеспечивает доступ к веб-интерфейсу и метрикам (HTTP, порт 8081) через доменное имя. Требуется установленного Ingress-контроллера (например, `nginx`).

Включите Ingress в манифесте тира:

```
tiers:
- name: default
  ingress:
    enabled: true
    host: picodata.example.com
    ingressClassName: nginx
    annotations:
      nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
  tls:
```

```
- secretName: picodata-tls
  hosts:
    - picodata.example.com
```

После применения манифеста веб-интерфейс будет доступен по адресу `https://picodata.example.com`.

Примечание

Ingress даёт доступ только к HTTP-интерфейсу (порт 8081). Для доступа по протоколу PostgreSQL (порт 5432) используйте `externalService`.

LoadBalancer / NodePort

Для доступа к PostgreSQL-интерфейсу снаружи кластера используйте поле `externalService`:

```
tiers:
  - name: default
    externalService:
      enabled: true
      type: LoadBalancer # или NodePort
```

При `type: LoadBalancer` Kubernetes создаёт отдельный объект `Service` с внешним IP. При `type: NodePort` PostgreSQL-интерфейс будет доступен на фиксированном порту каждого узла.

Получите внешний адрес:

```
kubectl get svc -n picodata -l picodata.io/service-type=external
```

Per-pod-сервисы для клиентов, работающих с шардированными данными

Драйверы `picodata-jdbc` и `picorun` учитывают распределение данных по шардам и умеют маршрутизировать запросы напрямую на под, хранящий нужный шард, без излишней нагрузки на сеть. Чтобы это работало, каждый под должен объявить **свой собственный** внешний адрес в `_pico_peer_address`.

Включите `perPod: true` в секции `externalService`:

```
tiers:
  - name: default
    externalService:
      enabled: true
      type: LoadBalancer
      perPod: true
```

Оператор создаст один `LoadBalancer Service` на каждый под тира. MetalLB (или cloud LB) назначит каждому `Service` уникальный VIP. Init-контейнер дожждётся назначения VIP и передаст его Picodata в качестве `advertise`-адреса — Picodata объявит адрес вида `{vip}:{port}` в таблице `_pico_peer_address`.

Требования

Необходим MetalLB (или облачный провайдер с поддержкой `LoadBalancer`) с пулом IP-адресов достаточного размера: минимум $N + 1$ адресов на тир (N подов + один tier-level Service).

Проверьте назначенные VIP и созданные Services:

```
kubectl get svc -n picodata -l picodata.io/external-pod-service=true
```

Проверьте адреса в таблице `_pico_peer_address`:

```
SELECT raft_id, address, connection_type
FROM _pico_peer_address
WHERE connection_type = 'pgproto';
```

raft_id	address	connection_type
1	192.168.1.201:5432	pgproto
2	192.168.1.202:5432	pgproto

Настройка MetalLB для per-pod VIP

L2-режим (простой, без BGP)

L2-режим работает без дополнительного сетевого оборудования: MetalLB отвечает на ARP-запросы за выбранным IP-адресом. Подходит для разработки и развёртывания небольших кластеров. Ограничение: весь трафик на VIP проходит через один узел («спикер»).

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: picodata-pool
  namespace: metallb-system
spec:
  addresses:
    - 192.168.1.200-192.168.1.229 # пул из ≥ N+1 адресов на тир
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: picodata-l2
  namespace: metallb-system
spec:
  ipAddressPools:
    - picodata-pool
```

BGP-режим (рекомендуется для production)

В BGP-режиме каждый узел анонсирует маршруты к VIP напрямую в сеть через протокол BGP. Преимущества перед L2:

- трафик распределяется по всем узлам через ECMP;
- failover быстрее (секунды по BGP keepalive vs десятки секунд ARP);
- не нужен ARP-прокси, работает в любой IP-сети.

Предварительные требования:

- маршрутизатор (или ToR-коммутатор) с поддержкой BGP;
- договорённые ASN: localASN (ASN кластера Kubernetes), peerASN (ASN роутера);
- IP-адрес BGP-пира со стороны роутера.

Конфигурация MetalLB:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: picodata-pool
```

```

namespace: metallb-system
spec:
  addresses:
    - 192.168.1.200-192.168.1.229 # пул из ≥ N+1 адресов на тир
  ---
apiVersion: metallb.io/v1beta2
kind: BGPPeer
metadata:
  name: upstream-router
  namespace: metallb-system
spec:
  myASN: 64512 # ASN кластера Kubernetes (private range: 64512–65534)
  peerASN: 64513 # ASN upstream-роутера
  peerAddress: 192.168.0.1 # IP BGP-пира (роутер / ToR-коммутатор)
  ---
apiVersion: metallb.io/v1beta1
kind: BGPAdvertisement
metadata:
  name: picodata-bgp
  namespace: metallb-system
spec:
  ipAddressPools:
    - picodata-pool
  peers:
    - upstream-router

```

После применения конфигурации MetalLB установит BGP-сессию с роутером и начнёт анонсировать маршруты к VIP из пула `picodata-pool`. Проверить сессию:

```
kubectl logs -n metallb-system -l component=speaker | grep -i bgp | tail -20
```

BGP-сессия должна перейти в состояние `Established`.

Несколько BGP-пиров

Если в кластере несколько ToR-коммутаторов, добавьте несколько ресурсов `BGPPeer` — MetalLB будет анонсировать маршруты каждому из них.

BGP через Cilium (альтернатива MetalLB)

Если кластер использует Cilium в качестве CNI, MetalLB не нужен: Cilium включает собственный **BGP Control Plane** и умеет назначать LoadBalancer IP из внутреннего пула, анонсируя маршруты через BGP.

Предварительные требования:

- Cilium ≥ 1.13 с включённым `bgpControlPlane.enabled=true`
- маршрутизатор с поддержкой BGP

Пул IP-адресов (CiliumLoadBalancerIPPool):

```

apiVersion: "cilium.io/v2alpha1"
kind: CiliumLoadBalancerIPPool
metadata:
  name: picodata-pool
spec:
  cidrs:
    - cidr: "192.168.1.200/27" # 30 адресов — достаточно для N подов + tier LB

```

BGP-политика (CiliumBGPPeeringPolicy):

```

apiVersion: "cilium.io/v2alpha1"
kind: CiliumBGPPeeringPolicy
metadata:
  name: picodata-bgp
spec:

```

```

nodeSelector:
  matchLabels:
    kubernetes.io/os: linux # применяется ко всем узлам
virtualRouters:
- localASN: 64512
  exportPodCIDR: false
  neighbors:
  - peerAddress: "192.168.0.1/32" # IP upstream-роутера
    peerASN: 64513
  serviceSelector:
    matchExpressions:
    - key: picodata.io/external-pod-service
      operator: In
      values: ["true"]
    - key: picodata.io/cluster
      operator: Exists

```

`serviceSelector` ограничивает анонс только per-pod сервисами Picodata. Это предотвращает утечку внутренних ClusterIP в BGP-таблицу роутера.

Проверить что Cilium установил BGP-сессию:

```
cilium bgp peers
```

Node	Local AS	Peer AS	Peer Address	Session State	Since	Received	Sent
k8s-node-01	64512	64513	192.168.0.1	Established	1h	12	15
k8s-node-02	64512	64513	192.168.0.1	Established	1h	12	15

Cilium 1.16+

В Cilium 1.16 появился новый BGP API (`CiliumBGPClusterConfig`, `CiliumBGPPeerConfig`, `CiliumBGPAdvertisement`). Показанный выше `CiliumBGPPeeringPolicy` поддерживается начиная с Cilium 1.13 и работает во всех версиях. Для новых установок смотрите документацию Cilium.

Shard-aware без LoadBalancer: DNS + BGP pod routing

Если CNI кластера анонсирует pod CIDR в сеть через BGP (Cilium или Calico в BGP-режиме), LoadBalancer-сервисы для shard-aware доступа **не нужны**. Pod IP становятся маршрутизируемыми напрямую — без MetalLB и без per-pod VIP.

В этом случае оператор работает в режиме по умолчанию: advertise-адрес каждого пода в `_pico_peer_address` — это стабильный FQDN:

```
hot-mycluster-1-0.hot-mycluster-interconnect.picodata.svc.cluster.local:5432
```

Shard-aware драйвер читает эти FQDN из `_pico_peer_address`, резолвит их в pod IP и подключается напрямую.

Что нужно:

1. **Pod IP маршрутизируемы снаружи** — CNI в BGP-режиме анонсирует pod CIDR в сеть, клиент может достучаться до `10.244.x.x` напрямую.

2. **Kubernetes DNS доступен с клиента** — один из вариантов:

- DNS forwarding: настроить на клиентской машине пересылку зоны `cluster.local` на IP сервиса kube-dns (если он доступен из сети);
- Делегирование зоны: сделать `cluster.local` поддоменом вашего DNS, делегировав его на CoreDNS кластера.

Что не нужно настраивать в PicoclusterDB:

```

tiers:
- name: hot
  externalService:
    enabled: true
    type: LoadBalancer # tier-level LB для bootstrap (опционально)
    # perPod: НЕ указывать – он не нужен

```

perPod: true и связанные per-pod Services не создаются. Оператор не выделяет IP из пула MetalLB на каждый под.

Схема подключения:

```

Клиент (вне кластера)
├── DNS: hot-mycluster-1-0.hot-mycluster-interconnect.picodata.svc.cluster.local
│   ├── forwarding *.cluster.local → CoreDNS кластера
│   └── ответ: 10.244.1.5 ← pod IP
└── TCP 10.244.1.5:5432
    └── маршрут через BGP: CNI анонсирует pod CIDR наружу

```

Ограничение

Подход работает только если клиент находится в той же L3-сети, куда CNI анонсирует pod CIDR. Для клиентов за NAT или через интернет по-прежнему нужны LoadBalancer-сервисы с публичными IP.

Управление плагинами**Сборка образа с плагином**

Стандартный образ Picodata не включает плагины. Для запуска плагина необходимо собрать кастомный Docker-образ, распаковав архив плагина в директорию shareDir (по умолчанию /usr/share/picodata).

Пример Dockerfile для плагина Radix:

```

FROM docker.binary.picodata.io/picodata:26.1.5 as release

USER root

RUN mkdir -p /usr/share/picodata

COPY radix_1.0.1-centos_el8.tar.gz /usr/share/picodata/radix.tar.gz

RUN cd /usr/share/picodata && tar -xvzf radix.tar.gz && rm radix.tar.gz

RUN chown -R picodata:picodata /usr/share/picodata

USER picodata

```

После сборки и публикации образа укажите его в манифесте кластера:

```

spec:
  image:
    repository: registry.example.com/picodata-with-radix
    tag: "26.1.2-radix-1.0.1"

```

Описание плагина в манифесте

Плагины объявляются на двух уровнях: кластера и тира.

На **кластерном уровне** задаётся версия плагина и параметры первичной миграции:

```
спес:
  cluster:
    shareDir: /usr/share/picodata # путь к директории с .so-файлами плагинов
    plugins:
      - name: radix
        version: "1.0.0"
        migrationContext:
          tier_for_db_0: "default"
          # ... остальные параметры
```

На **уровне тира** задаётся привязка сервисов плагина к тире и listener-порт:

```
спес:
  tiers:
    - name: default
      plugins:
        - name: radix
          services:
            - name: radix
              listenerPort: 8082
```

После применения манифеста оператор автоматически выполнит полный жизненный цикл установки: CREATE PLUGIN → ADD SERVICE TO TIER → SET migration_context → MIGRATE TO → ENABLE. Установка запускается только когда все поды тира готовы.

Статус установки плагина:

```
kubectl get picoclusterdb picodata -n picodata \
-o jsonpath='{.status.tiers[*].plugins}'
```

Обновление версии Picodata

Для обновления версии Picodata измените поле `спес.image.tag` в манифесте и примените его:

```
спес:
  image:
    tag: "picodata:26.2.0"

kubectl apply -f picodata-cluster.yaml
```

Оператор выполнит безопасное обновление:

1. Отключит (DISABLE) все включённые плагины, несовместимые с новым образом
2. Дождётся фиксации изменения в Raft-кластере
3. Обновит образ во всех StatefulSet-ax (rolling update)
4. Установит и включит плагины для новой версии

Примечание

PVC (данные) при обновлении образа не затрагиваются.

Ребутстрап инстанса

Ребутстрап применяется когда инстанс не может вернуться в кластер после потери данных или повреждения тома. Оператор исключает инстанс из кластера (expel), стирает его данные и позволяет ему заново присоединиться как новому члену репликасета.

Внимание!

Рекрутстрап возможен только если репликасет содержит более одного инстанса ($\text{replicationFactor} \geq 2$) и хотя бы один из оставшихся инстансов доступен.

Создайте ресурс `PicoclusterInstanceRebootstrap`:

```
rebootstrap.yamlapiVersion: picodata.picodata.io/v1
kind: PicoclusterInstanceRebootstrap
metadata:
  name: rebootstrap-1
  namespace: picodata
spec:
  clusterRef:
    name: picodata          # имя PicoclusterDB
    instanceName: default-picodata-1-0 # имя пода (инстанса)

kubectl apply -f rebootstrap.yaml
```

Следите за фазами выполнения:

```
kubectl get picoclusterinstancerebootstrap -n picodata -w
```

NAME	INSTANCE	PHASE	AGE
rebootstrap-1	default-picodata-1-0	Expelling	10s
rebootstrap-1	default-picodata-1-0	Wiping	30s
rebootstrap-1	default-picodata-1-0	Rejoining	45s
rebootstrap-1	default-picodata-1-0	Succeeded	2m

После завершения ресурс `PicoclusterInstanceRebootstrap` можно удалить.

Резервное копирование

Экспериментальная функция

Резервное копирование и восстановление находятся в стадии эксперимента. Не используйте их в production без предварительного тестирования.

Настройка хранилища резервных копий

Добавьте в манифест кластера секцию `backup` с описанием тома для хранения резервных копий:

```
спес:
  backup:
    mountPath: /pico/backup # путь внутри пода, куда монтируется том
    volume:
      # Для production используйте NFS или другой тип тома с ReadWriteMany.
      # hostPath подходит только для одноузловых кластеров (minikube).
      nfs:
        server: nfs.example.com
        path: /exports/picodata-backup
```

Ограничения резервного копирования

- Резервная копия запускается только если **все поды кластера готовы** (1/1 Running). При недоступности хотя бы одного пода бэкап перейдёт в фазу Failed
- Для одного кластера **одновременно может выполняться только один** `PicoclusterBackup`.
- Хранилище резервных копий должно быть **доступно на запись** со всех узлов кластера. Для multi-node-окружений используйте NFS или другой том с поддержкой

ReadWriteMany. Оператор проверяет доступность тома перед запуском бэкапа (preflight-проверка)

- При **перезапуске пода** в ходе бэкапа консистентность резервной копии не гарантируется. Факт перезапуска фиксируется в `status.podSnapshots` для последующей диагностики
- По истечении `spec.backupTimeout` оператор прерывает операцию через `pico.abort_ddl()`. Прерванный бэкап **не пригоден для восстановления**

Создание резервной копии

Создайте ресурс `PicoclusterBackup`:

```
backup.yamlapiVersion: picodata.picodata.io/v1
kind: PicoclusterBackup
metadata:
  name: backup-1
  namespace: picodata
spec:
  clusterRef:
    name: picodata          # имя PicoclusterDB
    backupTimeout: 3600     # максимальное время ожидания (секунды)

kubectl apply -f backup.yaml
```

Следите за статусом:

```
kubectl get picoclusterbackup -n picodata -w
```

NAME	CLUSTER	STATUS	AGE
backup-1	picodata	Succeeded	5m

Путь к сохранённой резервной копии:

```
kubectl get picoclusterbackup backup-1 -n picodata \
  -o jsonpath='{.status.backupDir}'
```

Ограничения восстановления

- Восстановление — **деструктивная операция**: оператор останавливает весь кластер (scale до 0), стирает данные каждого инстанса и заменяет их содержимым снапшота
- Для одного кластера **одновременно может выполняться только одна** операция восстановления
- При `backupRef` ссылаемый `PicoclusterBackup` должен быть в фазе `Succeeded`.
- Если восстановление завершилось с ошибкой, **кластер остаётся остановленным** (все `StatefulSet` масштабированы до 0) для ручного разбора причин. Автоматического отката нет.
- Хранилище резервных копий должно быть **смонтировано в те же поды**, что и при создании бэкапа: оператор ожидает структуру `<mountPath>/<instanceName>/<backupDir>/`

Восстановление из резервной копии

Создайте ресурс `PicoclusterRestore`:

```
restore.yamlapiVersion: picodata.picodata.io/v1
kind: PicoclusterRestore
metadata:
  name: restore-1
```

```
namespace: picodata
спес:
  clusterRef:
    name: picodata          # имя PicoclusterDB для восстановления
  backupRef:
    name: backup-1          # ссылка на ресурс PicoclusterBackup
```

Если ресурс PicoclusterBackup был удалён, укажите директорию напрямую:

```
спес:
  clusterRef:
    name: picodata
  backupDir: "20260508T143720" # имя поддиректории в томе резервных копий
kubectly apply -f restore.yaml
```

Anti-affinity

По умолчанию оператор автоматически добавляет правило podAntiAffinity, которое запрещает размещение двух подов одного репликасета на одном узле Kubernetes. Это гарантирует отказоустойчивость при потере узла.

Требование к StorageClass

Anti-affinity работает корректно только если StorageClass использует volumeBindingMode: WaitForFirstConsumer.

При режиме Immediate (значение по умолчанию во многих дистрибутивах) PVC создаются и привязываются к узлам **до** планирования подов. В результате оба PVC одного репликасета могут оказаться на одном узле, и поды не смогут «разойтись» — один из них зависнет в состоянии Pending.

Проверьте режим вашего StorageClass:

```
kubectly get storageclass -o custom-columns=NAME:.metadata.name,BINDING:.volumeBindingMode
```

Если значение Immediate, создайте StorageClass с нужным режимом:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: picodata-sc
provisioner: <ваш-provisioner> # например, rancher.io/local-path
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Retain
```

И укажите его в тире:

```
tiers:
- name: default
  storage:
    size: 10Gi
    storageClassName: picodata-sc
```

Примечание

В управляемых кластерах (GKE, EKS, AKS) стандартные StorageClass обычно уже используют WaitForFirstConsumer. Проблема наиболее характерна для self-hosted-кластеров с local-path, Longhorn или OpenEBS.

Отключение anti-affinity

Для развёртывания на одноузловом кластере (minikube, dev-окружение) отключите anti-affinity для нужного тира:

```
tiers:
- name: default
  disableAutoAntiAffinity: true
```

Внимание!

Не отключайте anti-affinity в production: при потере узла могут упасть сразу несколько экземпляров одного репликасета.

PodDisruptionBudget

Оператор автоматически создаёт PodDisruptionBudget для тиров с фактором репликации ≥ 3 . PDB ограничивает количество одновременно недоступных подов при плановых операциях (drain-узлы, обновление кластера Kubernetes), сохраняя кворум кластера.

При необходимости выполнить drain на кластере без свободных узлов — когда Kubernetes не может переместить поды — временно отключите PDB:

```
tiers:
- name: default
  disablePDB: true
```

После завершения drain верните значение в false (или удалите поле).

Внимание!

Не оставляйте PDB отключённым в production. Без PDB плановое обслуживание узлов может нарушить кворум кластера.

Удаление кластера

```
kubectl delete picoclusterdb picodata -n picodata
```

Удаление пользовательского ресурса (CR) автоматически удаляет все связанные с ним ресурсы (StatefulSet, Service, ConfigMap) через механизм ownerReference.

Внимание!

PVC (постоянные тома с данными) при удалении CR **не удаляются** — это защита от случайной потери данных. Для полного удаления данных выполните: `kubectl delete pvc -n picodata --all`

Мониторинг

Каждый экземпляр Picodata отдаёт метрики в формате Prometheus на порту 8081, путь /metrics.

ServiceMonitor через CR

Если в кластере установлен Prometheus Operator, включите автоматический сбор метрик в манифесте кластера:

```
spec:
  serviceMonitor:
    enabled: true
    interval: "30s"
    labels:
      release: kube-prometheus-stack # должен совпадать с serviceMonitorSelector вашего Prometheus
```

Оператор создаёт по одному ServiceMonitor на каждый тип — Prometheus начнёт получать данные от каждого инстанса через headless-сервис.

PodMonitor с job-лейблом по имени кластера

Для использования с Grafana-дашбордом рекомендуется настроить PodMonitor так, чтобы label job в Prometheus соответствовал имени кластера. Это позволяет выбирать кластер в выпадающем меню дашборда:

```
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: picodata
  namespace: picodata
  labels:
    release: kube-prometheus-stack
spec:
  jobLabel: app.kubernetes.io/instance # job = clusterName
  selector:
    matchExpressions:
      - key: app.kubernetes.io/instance
        operator: Exists
  namespaceSelector:
    matchNames:
      - picodata
  podMetricsEndpoints:
    - port: http
      path: /metrics
      interval: 30s
```

Один ресурс охватывает все кластеры в пространстве имён (namespace) и не требует изменений при добавлении новых кластеров.

Grafana-дашборд

Официальный дашборд Picodata доступен в репозитории `core/picodata:monitoring/dashboard/Picodata.json`

Импортировать через UI Grafana: **Dashboards** → **Import** → **Upload JSON file** или через URL файла.

После импорта выберите datasource Prometheus и нужный кластер в выпадающем меню job.

Настройка серверов для кластера

В данном разделе приведены рекомендации по настройке ОС и системного ПО на серверах, на которых предполагается запуск инстансов Picodata. Рекомендации помогут избежать проблем, связанных с промышленной эксплуатацией и мониторингом кластера.

Эксплуатация и мониторинг

Настройка службы времени

Поскольку кластер Picodata — это распределенная система, то необходимо, чтобы все серверы кластера жили по единому времени.

Рекомендация: настроить на всех серверах службу времени NTP или Chrony, убедиться в отсутствии разбега времени.

Настройка DNS

Имя сервера более наглядно для восприятия администратором, чем IP-адрес. К тому же, при использовании DHCP или при замене сервера, IP-адрес может измениться, в отличие от DNS-имени.

Рекомендация: использовать на всех серверах имя (FQDN — fully qualified domain name) вместо IP-адреса.

Настройка журналирования

В файлах журналов содержится много полезной информации, особенно если в кластере начинают происходить непонятные сбои, поэтому за журналами нужно следить и своевременно их ротировать. В момент сбоев возможны резкие всплески количества сообщений, что может приводить к недостатку места на диске. Стоит организовать хранение журналов на выделенном дисковом разделе, отдельно от всего остального (система, данные) и обязательно следить за свободным местом на этом разделе. Рекомендуем настроить сохранение логов в `journald`, а не в файлы. Менеджер загрузки `Systemd` умеет самостоятельно проводить ротацию журналов с учетом свободного места на разделе.

Рекомендация: выделить отдельный и достаточной большой раздел диска для файлов журналов; использовать `journald` или самостоятельно настроить ротацию файлов журналов логов; настроить оповещения (alerts) в системе мониторинга для контроля за свободным местом на разделе.

Настройка SELinux

На данный момент Picodata не поддерживает правила SELinux.

Рекомендация: отключить SELinux (установите `SELINUX=disabled` в файле `/etc/selinux/config` и перезагрузитесь).

Настройка мониторинга

Серверы должны быть подключены к системе мониторинга — это поможет своевременно отслеживать проблемы при их эксплуатации. Следует следить как за метриками, так и за журналами инстансов. Помимо мониторинга серверов, обязательно нужно отслеживать метрики кластера Picodata.

Рекомендация: настроить мониторинг в Grafana — это позволит находить проблемы в работе кластера и своевременно на них реагировать — а также мониторинг метрик самих серверов, как минимум, CPU, RAM, HDD.

Настройка доступности портов

Picodata — это распределенный кластер, инстансы которого общаются между собой по бинарному протоколу (iproto). В то же время, клиентам необходим доступ к прикладным протоколам: клиентской консоли, совместимой с PostgreSQL, портам, необходимым плагинам для работы с данными, портам для мониторинга кластера по HTTP — важно обеспечить доступность всех этих портов с помощью настройки межсетевого экрана.

Рекомендация: обеспечить сетевую доступность серверов как между собой (по бинарному протоколу), так и доступ клиентов до нужных протоколов (HTTP, PostgreSQL, порты плагинов).

Настройка сохранения дампов памяти при сбоях

Информация из coredump-файлов поможет разработчикам Picodata решать проблемы в случае сбоя кластера.

Рекомендация: включить сборку coredump-файлов на серверах.

Пример включения coredump

добавить строку в файл /etc/security/limits.conf: * soft core unlimited
добавить в файл /etc/sysctl.conf строки: kernel.core_pattern = /tmp/core.%s.%e.%p.%h.%t
fs.suid_dumpable = 2

Настройка лимитов для ресурсов

В ряде случаев корректной работе Picodata может препятствовать ограничение на число дескрипторов файловой системы. Чтобы избежать этого, следует явно задать максимальное значение для соответствующих параметров для пользователя, под которым будут работать процессы picodata.

Рекомендация: создать файла/etc/security/limits.d/99-picodata.conf со следующим содержанием (для пользователя picodata):

picodata	soft	nofile	65535
picodata	hard	nofile	65535
picodata	soft	core	unlimited
picodata	hard	core	unlimited

Настройка Journald

При регистрации системных событий ОС с помощью службы Journald (используется по умолчанию в Systemd) может возникнуть ситуация, когда журналы службы хранятся только в ОЗУ (`/run/log/journal`) и из-за этого со временем становятся недоступны.

Рекомендация: настроить Journald так, чтобы журналы явно сохранялись на диске. Для этого добавьте в файл `/etc/systemd/journal.conf` строку `Storage=persistent`.

Безопасность

Настройка пользователей

Инстанс Picodata можно запустить под любым пользователем, включая `root`, но с точки зрения безопасности это неправильно, поэтому мы рекомендуем создать отдельного пользователя с одинаковым UID и GID на всех серверах кластера, например с именем `picodata`, и запускать все процессы, связанные с работой Picodata, под этим пользователем.

Рекомендация: создать на всех серверах пользователя `picodata` с одноименной группой (`picodata:picodata`) или другого пользователя, под которым будут работать процессы кластера.

Настройка аудита

Аудит событий поможет понять действия пользователей, если, например, в базе неожиданно пропадет информация. Сообщения аудита можно сохранять в файлы, `journald` или же в отдельный `pipe`, который будет их сразу пересылать во внешнюю систему хранения. Несмотря на то, что сообщений аудита не так много, тем не менее, необходимо настроить их ротацию и следить за местом на разделе, где хранятся файлы аудита.

Рекомендация: включить журналы аудита *при развёртывании кластера*; использовать развёртывание с помощью нашей роли Ansible (в этом случае аудит будет включён по умолчанию).

Производительность

Настройка дисковой подсистемы

В некоторые моменты времени, например при сбоях, в файлы журналов может записываться большое количество информации, что негативно скажется на других операциях кластера: WAL-журналы, снимки, данные БД при использовании движка Vinyl. Несмотря на то, что современные диски быстрые, для получения максимальной производительности системы ввода-вывода стоит разнести файлы журналов и данные по разным дисковым устройствам.

Рекомендация: разнести разделы с журналами и данными по разным дисковым устройствам.

Настройка swap

Большая часть информации, даже при использовании движка Vinyl, обрабатывается в ОЗУ. Если в системе включена подкачка (swap), то может сложиться ситуация, когда ОС будет пытаться сохранить страницы памяти на диск, что приведет к снижению производительности кластера.

Рекомендация: отключить подкачку (swap) на серверах.

Установка Picodata

Выбор версии

Если иное не оговорено, используйте релизную сборку Picodata, установленную из подготовленного разработчиком пакета. Такие сборки поставляются с поддержкой отладочных символов, что упрощает разбор инцидентов.

Рекомендация: использовать Picodata из предоставляемых на официальной странице загрузки пакетов.

Выбор топологии

Для повышения отказоустойчивости кластера имеет смысл использовать отдельный дополнительный тир, инстансы которого будут поддерживать Raft-кворум, но не будут подвергаться нагрузке по обработке данных шардированных таблиц. Подробнее эта концепция описана в статье Raft и отказоустойчивость.

Рекомендация: выделить отдельный тир-арбитр с параметрами `replication_factor=1` и `can_vote: true` и назначить ему 5 репликasetов с размещением в разных физических локациях. Для остальных тиров использовать параметр `can_vote: false`.

Предоставление данных

Для диагностики, разбора инцидентов и оценки производительности Picodata в условиях промышленной эксплуатации передайте разработчику подробный набор сведений о программном и аппаратном обеспечении, на котором работает кластер.

Рекомендация: предоставить разработчику следующий набор сведений:

- версия ОС, версия Picodata, версии всех установленных плагинов
- карта инсталляции кластера:
 - в случае развертывания через Ansible достаточно предоставить плейбук и инвентарный файл
 - в случае развертывания через Helm Chart предоставьте версия Chart, а также использованные параметры (values)
 - в случае ручного развертывания предоставьте список узлов, топологию (физическую, виртуальную, контейнерную), список настроек каждого узла (data dir, log dir, config dir), порты, на которых подняты интерфейсы IPROTO, PostgreSQL и HTTP
- расположение coredump-файлов

Управление кластером в промышленной среде с ограниченными привилегиями

В данном разделе приведена информация по установке кластера Picodata в промышленной среде, в которой действуют ранее настроенные политики безопасности. Информация дополняет инструкцию по использованию роли `picodata-ansible`.

Общие сведения

В крупных организациях роли системного администратора (Администратора ОС) и Администратора БД разделены с целью обеспечения информационной безопасности. В этом случае роль позволяет выполнить первоначальную настройку серверов с учетной записью Администратора ОС и последующую инсталляцию кластера Администратором БД.

Независимо от варианта инсталляции кластера процессы инстансов на серверах могут быть запущены под сервисной учетной записью (см. параметры `user` и `group`).

Требования для установки кластера

Для удобства администрирования кластера корневые директории желательно (но не обязательно) располагать в отдельном разделе (`/opt`, `/data`, `/app`).

На серверах кластера должны быть выполнены действия:

- установлен пакет `picodata` необходимой версии (его можно установить на шаге Первоначальная настройка серверов)
- должна присутствовать утилита `setfacl` (обычно находится в пакете `acl`)
- создана учетная запись для Администратора ОС (с полными `sudo`-полномочиями без запроса пароля), например `sysadm`
- создана учетная запись для Администратора БД, например `dbadm`
- создана учетная запись для пользователя, указанного в параметре `user`, например `picodata`. Администратор БД должен иметь возможность переключения на эту учетную запись без запроса пароля (`sudo -su picodata`)

Параметры создания учетной записи, указанной для `user`:

```
create_home=yes
shell=/sbin/nologin
```

Для примера, настройка правил `sudo` для возможности переключения на учетную запись `picodata` из-под пользователя `dbadm`:

```
Cmd_Alias SHELL = /bin/sh, /bin/bash"
dbadm ALL = (picodata) NOPASSWD: SHELL
```

В инвентарном файле:

- определите директории: `conf_dir`, `data_dir`, `run_dir`, `log_dir`, `share_dir`
- выставите переменные:
`user: picodata`

```
group: picodata
rootless: true
```

Настройки для Systemd

Рекомендуем использовать именно этот вариант для установки кластера.

Первоначальная настройка серверов

Выполняется под учетной записью Администратора ОС, например sysadm.

Первоначальная настройка серверов состоит из следующих шагов:

- при отсутствии на серверах пакета picodata подключите репозиторий Picodata и установите из него пакет picodata последней доступной версии; если отсутствует доступ в интернет, то в переменной picodata_package_path можно указать путь до пакета Picodata для установки на серверах, при этом пакет должен находиться на станции управления Ansible
- создайте директории для работы picodata
- создайте службу systemd для supervisord (при выборе системы управления процессами Supervisord)
- при выводе журналов в файлы, создайте logrotate-файл для их подрезки

Здесь и далее в примерах плейбук picodata.yml взят из руководства по роли Ansible.

```
---
- name: Deploy picodata cluster
  hosts: all
  become: true

  tasks:

    - name: Import picodata-ansible role
      ansible.builtin.import_role:
        name: picodata-ansible
```

Выполните команду для подготовки серверов:

```
ansible-playbook -i <inventory> picodata.yml -t deploy_become -u sysadm -k
```

Инсталляция кластера

Выполняется под учетной записью Администратора БД, например dbadm.

Установите кластер командой:

```
ansible-playbook -i <inventory> picodata.yml -u dbadm --become-user=picodata -k
```

При этом задачи, требующие повышенных привилегий, будут пропущены.

Рекомендация

Используйте доступ к серверам по ssh-ключу вместо пароля.

Управление кластером без повышения привилегий

Администратор БД может работать с процессами инстансов при переключении на пользователя, указанного в параметре user.

После переключения на пользователя `user` для работы `systemctl` выставите переменную окружения `XDG_RUNTIME_DIR` командой:

```
export XDG_RUNTIME_DIR=/run/user/${id -u}
```

Примеры команд для управления процессами инстансов на серверах кластера, которые может выполнять Администратор БД (без `sudo`):

```
systemctl --user --all status test@*
systemctl --user restart test@default-1000
systemctl --user -s HUP kill test@router-1001
systemctl --user --all stop test@*
```

Где:

- `cluster_name = test`
- `default-1000`, `router-1001` — названия инстансов кластера

Удаление кластера

Удаление кластера выполняется в обратном порядке: сначала под учетной записью Администратора БД (например `dbadm`) остановите кластер и удалите данные, затем, при необходимости, под Администратором ОС удалите системные файлы и директории.

Для удаления кластера выполните команду:

```
ansible-playbook -i <inventory> picodata.yml -t remove -u dbadm --become-user=picodata -k
```

Очистка серверов от системных файлов кластера

Выполняется под учетной записью Администратора ОС, например `sysadm`.

Для очистки серверов выполните команду:

```
ansible-playbook -i <inventory> picodata.yml -t remove_become -u sysadm -k
```

При этом будут удалены все корневые директории, которые были созданы для кластера.

Настройки для Supervisord

На серверах:

- установите пакет `supervisor`

В инвентарном файле:

- при необходимости определите директории: `supervisord_dir`
- выставите значение переменной `init_system` в `supervisord`

Первоначальная настройка серверов, инсталляция и удаление кластера проводятся теми же командами, как и для `supervisord`.

Работа с кластером через Supervisord

Примеры команд для управления процессами инстансов на серверах кластера, которые может выполнять Администратор БД:

```
supervisorctl -c /data/picodata/supervisord/test.conf status
supervisorctl -c /data/picodata/supervisord/test.conf restart default-1000
```

```
supervisorctl -c /data/picodata/supervisord/test.conf signal HUP router-1001
supervisorctl -c /data/picodata/supervisord/test.conf stop all
```

Где:

- `supervisor_dir = /data/picodata/supervisord`
- `cluster_name = test`
- `default-1000`, `router-1001` — названия инстансов

Для удобства администратора можно установить псевдоним (`alias`) в командной оболочке:

```
picosuper='supervisorctl -c /data/picodata/supervisord/test.conf'
```

и в дальнейшем использовать его при вызове команд:

```
picosuper status
picosuper restart default-1000
picosuper signal HUP router-1001
picosuper stop all
```

Обновление кластера

Обновление кластера поддерживается в двух режимах:

- с остановкой всех экземпляров кластера (более надёжный и рекомендованный способ)
- без остановки кластера, т.е. с поэтапным переводом экземпляров на новую версию продукта (более рискованный способ, но позволяющий избежать простоя)

В обоих случаях после обновления версии Picodata в кластере могут различаться максимум на 1 минорное значение. Например, в кластере могут одновременно работать экземпляры с Picodata 25.4 и 25.5.

Важно

Перед обновлением кластера сделайте его резервную копию — она пригодится для отката кластера к первоначальному состоянию в случае проблем обновления.

Обновление с остановкой кластера

Обновление кластера этим способом более предпочтительно, так как происходит быстрее и предсказуемее. Процедура сводится к:

- организации запланированного окна обслуживания, в течение которого кластер будет недоступен для клиентов
- отключению всех плагинов в кластере (при их наличии)
- остановке всех экземпляров кластера
- установке новой версии ПО (см. ниже)
- поэтапному запуску экземпляров уже на новой версии ПО
- включению плагинов (при их наличии)

Отключение плагинов

Если в кластере включены плагины, то перед обновлением их необходимо отключить:

- определите состав плагинов и их версии (`SELECT * FROM _pico_plugin;`)
- для каждого плагина выполните команду вида `ALTER PLUGIN plugin_name 0.0.1 DISABLE;`, указав корректные имя и версию

Остановка экземпляров

См. раздел Завершение работы и обработка сигналов.

Обновление ПО

При использовании Picodata в промышленной среде следует обновить поставляемую сборку продукта. Если Picodata была установлена из готового пакета для определенной ОС, то следует обновить этот пакет. При наличии подключенного репозитория Picodata обновленный пакет будет доставлен в рамках общего обновления системы (`sudo apt-get upgrade` для систем с Apt и `sudo dnf update` для систем с Yum/DNF). Если пакет был установлен вручную, без подключения репозитория Picodata, то следует воспользоваться системными средствами для управления индивидуальными пакетами (`dpkg/rpm`).

Обновление Picodata из исходного кода

Если Picodata была собрана из исходного кода, то необходимо обновить данные локальной копии Git-репозитория (`git pull && git submodule update --init --recursive`), очистить данные предыдущей сборки (`cargo clean`) и собрать новую версию бинарного файла `picodata`. Важно удостовериться, что после этого будет запускаться именно новая версия `picodata`.

Запуск после обновления

После обновления следует перезапустить инстансы Picodata с прежними параметрами. Рекомендуется в первую очередь запустить голосующие инстансы, затем — остальные.

Обновление и включение плагинов

Плагины должны быть скомпилированы под ту же версию продукта, под которой их планируется запускать.

Ниже приведён порядок действий для обновления отдельного плагина в кластере. Если плагинов несколько, то указанные шаги следует выполнить для каждого плагина.

1. Отключите плагин и выключите инстанс. Поместите файлы плагина в директорию, заданную для параметра `instance.share_dir`. Если при пересборке плагина его версия не менялась, то замените файлы старой версии. Повторите эти действия для всех инстансов кластера.
2. Запустите кластер с новой версией Picodata.
3. Выполните действия по включению плагина. Пример необходимых команд: `CREATE PLUGIN plugin_name 0.2.0; ALTER PLUGIN plugin_name 0.2.0 ADD SERVICE plugin_service_name TO TIER tier_name; ALTER PLUGIN plugin_name MIGRATE TO 0.2.0; ALTER PLUGIN plugin_name 0.2.0 ENABLE;`
4. Если у плагина есть настраиваемые параметры, которые необходимо задать, используйте команду `picodata plugin configure`. Пример: `picodata plugin configure --peer andy@127.0.0.1:3001 --service-password-file pass.txt plugin_name 0.2 share_dir/plugin_config.yaml`

Обновление без остановки кластера

Для обновления кластера без остановки достаточно установить новую версию Picodata на каждый узел и последовательно перезапустить инстансы.

Действия после обновления кластера

Проверка гетерогенности кластера

После завершения процедуры обновления проверьте, что все инстансы кластера перешли на новую версию и находятся в состоянии Online. Пример:

```
SELECT name, picodata_version,current_state FROM _pico_instance;
+-----+-----+-----+
| name | picodata_version | current_state |
+-----+-----+-----+
| i1   | 25.5.0-1566-g7f2b88e29 | ["Online", 10] |
+-----+-----+-----+
| i2   | 25.5.0-1566-g7f2b88e29 | ["Online", 6]  |
+-----+-----+-----+
| i3   | 25.5.0-1566-g7f2b88e29 | ["Online", 7]  |
+-----+-----+-----+
| i4   | 25.5.0-1566-g7f2b88e29 | ["Online", 11] |
+-----+-----+-----+
(4 rows)
```

Эту информацию можно посмотреть и через веб-интерфейс Picodata.

Если часть инстансов находятся в состоянии Offline и/или остались на прежней версии, проверьте сетевую доступность соответствующих узлов и попробуйте перезапустить инстансы. Если это не поможет, удалите такие инстансы из кластера вручную (см. ниже).

Проверка применения нового набора системных таблиц

Новая функциональность, поставляемая с актуальной версией Picodata, может отражаться на составе системных таблиц (глобальные таблицы с префиксом _pico, хранящие настройки СУБД). При обновлении кластера, изменения полей в системных таблицах вступят в силу после перехода всех инстансов кластера на новую версию Picodata. Для отслеживания этого используйте следующую команду, которая действует глобально в рамках кластера:

```
SELECT value FROM _pico_property WHERE key = 'system_catalog_version';
```

Значение 'system_catalog_version' обязательно меняется при обновлении на новую мажорную версию Picodata. Сравнивая это значение до и после обновления, вы можете определить, что кластер успешно обновился.

Удаление инстансов из кластера вручную

При обновлении кластера без остановки возможна ситуация, когда в raft-группе продолжают числиться инстансы, которых фактически уже нет (например, из-за изменения UUID). Для удаления таких инстансов из кластера вручную понадобится команда `expe1` и UUID этих инстансов. Значения UUID можно получить следующим образом: `picodata status --peer admin@<binary-адрес одного из новых инстансов>`. Пример вызова `expe1`: `picodata expe1 --peer admin@<binary-адрес одного из новых инстансов> <UUID инстанса, который нужно удалить> --force`. Это команда потребует пароль администратора СУБД. Если пароль не был установлен, то подключитесь к кластеру командой `picodata admin` и установите его: `ALTER USER admin WITH PASSWORD '<my secret pass>'`.

Откат обновления

Если обновление прошло неуспешно, попробуйте его откатить: остановите кластер и затем восстановите его из резервной копии.

Тестирование производительности

В данном разделе приведена информация о тестировании производительности Picodata.

Для замера производительности используется утилита Pgbench, которая подходит как для PostgreSQL, так и Picodata. Основой теста выступает TPC-B, используемый в Pgbench по умолчанию.

Системные требования

Убедитесь, что в вашей системе установлены следующие компоненты:

- uv (система управления зависимостями Python и его версиями)
- Pgbench (инструмент тестирования СУБД)

При необходимости установите отсутствующие компоненты с помощью пакетного менеджера ОС.

Использование стандартного теста TPC-B

Предварительные действия

Для установки и запуска стандартного теста TPC-B требуется работающий кластер Picodata.

Также, необходимо иметь в кластере пользователя postgres, обладающего правами на работу с таблицами. Пример создания и настройки пользователя: `CREATE USER postgres WITH PASSWORD 'Passw0rd'; GRANT CREATE TABLE TO postgres; GRANT READ TABLE TO postgres; GRANT WRITE TABLE TO postgres;`

Установка, инициализация и запуск теста

Файлы теста находятся в директории benchmark/tpcb в git-репозитории Picodata.

1. Откройте окно/вкладку терминала в корневой директории исходного кода Picodata и инициализируйте тест: `uv run python benchmark/tpcb/init.py \`

`"postgres://postgres:Passw0rd@127.0.0.1:4327?sslmode=disable" \ --scale 10`

2. Запустите тест: `pgbench \`

`"postgres://postgres:Passw0rd@127.0.0.1:4327?sslmode=disable" \ --scale 10 \ --time 30 \ --client 200 \ --protocol prepared \ --jobs 1 --progress 1 --no-vacuum`

Изменение параметров теста

При необходимости задайте собственные параметры для приведённой выше команды pgbench:

- данные пользователя. Укажите имя и пароль пользователя, под которым вы подключаетесь к экземпляру Picodata
- адрес подключения. В примере выше используется инстанс, принимающий подключения по протоколу PostgreSQL по адресу 127.0.0.1:4327. Убедитесь, что в

вашем случае указано актуальное значение этого параметра (см. `instance.pgproto.listen`)

- коэффициент масштабирования (`--scale 10`). Используйте значение, указанное при инициализации теста. При использовании более высокого значения `scale`, в кластере может возникнуть ошибка выделения памяти. Для обхода этой проблемы следует увеличить значение параметра `instance.memtx.memory` или `instance.vinyl.memory` (в зависимости от используемого движка)
- длительность теста (`--time 30`). Для снижения погрешностей иногда есть смысла увеличить время
- количество одновременных клиентов (`--client 200`). В зависимости от сценария, оптимальное значение может меняться в большую или меньшую сторону
- использовать подготовленные запросы (prepared statements, `--protocol prepared`). Этот режим позволяет добиться максимальной производительности теста

Примечание

Описание всех параметров можно найти в официальной документации Pgbench

Использование собственных тестов

Утилита Pgbench позволяет тестировать произвольные SQL-запросы, переданные в параметре `--file`.

Пример запуска теста с явным указанием скрипта:

```
pgbench \
  "postgres://postgres:Passw0rd@127.0.0.1:4327?sslmode=disable" \
  --file script.sql \
  --scale 10 \
  --time 30 \
  --client 200 \
  --protocol prepared \
  --jobs 1 --progress 1 --no-vacuum
```

Пример скрипта по умолчанию для TPC-B:

```
script.sql\set aid random(1, 100000 * :scale)
\set bid random(1, 1 * :scale)
\set tid random(1, 10 * :scale)
\set delta random(-5000, 5000)
BEGIN;
UPDATE pgbench_accounts SET abalance = abalance + :delta WHERE aid = :aid;
SELECT abalance FROM pgbench_accounts WHERE aid = :aid;
UPDATE pgbench_tellers SET tbalance = tbalance + :delta WHERE tid = :tid;
UPDATE pgbench_branches SET bbalance = bbalance + :delta WHERE bid = :bid;
INSERT INTO pgbench_history (tid, bid, aid, delta, mtime) VALUES (:tid, :bid, :aid, :delta, CURRENT_TIMESTAMP);
END;
```

Примечание

При использовании собственного скрипта может потребоваться изменить содержимое файла `init.py` или предусмотреть собственный способ инициализации теста. Файлы поставляемого с Picodata теста производительности находятся в директории `benchmark/tpcb` в git-репозитории Picodata.

Резервное копирование и восстановление

Данный раздел описывает способы резервного копирования и восстановления из резервной копии в Picodata.

Общие сведения

Объектами резервного копирования в Picodata являются:

- файлы конфигурации, необходимые для запуска кластера Picodata
- данные, которые хранятся на инстансе: копия всех глобальных таблиц БД (включая конфигурацию СУБД, БД, хранимые процедуры, пользовательские объекты) и часть шардированных таблиц БД (пользовательских таблиц, сегментированных по заданным колонкам)

При резервном копировании мы различаем работу со всем кластером и работу с отдельными инстансами.

Копирование всего кластера

Наиболее предпочтительным является резервное копирование кластера целиком: это обеспечивает наиболее беспроblemное и консистентное восстановление СУБД.

Для этой цели в Picodata доступны два способа: с помощью SQL и через роль Ansible.

С помощью SQL

DDL-команда BACKUP используется для создания согласованной, кластерной резервной копии данных:

```
Создание резервной копииBACKUP OPTION (TIMEOUT = 3.0);
```

По умолчанию, резервные копии сохраняются в <instance-dir>/backup.

Для восстановления из ранее созданной резервной копии используйте CLI-команду `picodata restore`:

```
Восстановление из резервной копииpicodata restore --path <backup-path>
```

Более подробно команда BACKUP описана в нашем Справочнике SQL.

С помощью Ansible

Для данного метода используйте роль Picodata для Ansible, где создание резервной копии и восстановление из нее происходит простым вводом соответствующей команды:

```
Создание резервной копииansible-playbook -i hosts.yml picodata.yml -t backup
```

```
Восстановление из резервной копииansible-playbook -i hosts.yml picodata.yml -t restore
```

Копирование отдельных инстансов

Резервное копирование отдельных инстансов не требуется в том случае, если в кластере настроена репликация (т.е. значение параметра `replication_factor` превышает 1). При выходе из строя одного инстанса, его данные останутся в кластере на оставшихся инстансах того же репликасета. Для возврата инстанса в работу потребуется сначала исключить его прежний UUID из репликасета и затем запустить его без каких-либо данных — после присоединения обратно к репликасету данные автоматически скопируются на него.

Создание локальной резервной копии

При выходе из строя инстанса, данные которого не реплицированы (либо в ситуации, когда недоступны все реплики в репликасете), кластер может продолжить работу, однако шардированные данные, хранившиеся на таком инстансе, не будут доступны. Эти данные, а также данные глобальных таблиц, оведущиваются в виде файлов `*.xlog`, `*.vlog` и `*.snap` в рабочей директории инстанса. Имея резервную копию этих файлов, можно перезапустить инстанс под прежним UUID и вернуть его в состав `raft`-группы.

Резервное копирование файлов инстанса может быть выполнено следующей командой:

```
rsync -r <instance_dir> <backup_dir>
```

где:

- `<instance_dir>` — рабочая директория инстанса, указанная при запуске в параметре `picodata run --instance-dir`
- `<backup_dir>` — целевая директория для хранения резервной копии

Восстановление из локальной резервной копии

Восстановление данных инстанса из резервной копии выполняется той же командой `rsync`, но в обратную сторону:

```
rsync -r <backup_dir> <instance_dir>
```

Примечание

Восстановление из резервной копии следует проводить после остановки инстанса, чтобы избежать возможной неконсистентности данных.

Примечание

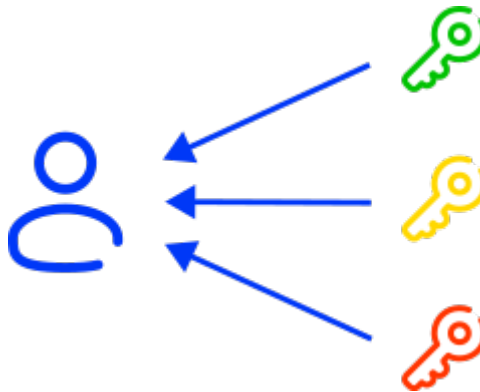
При работе в защищенной ОС информацию из этого раздела следует использовать совместно с документацией ОС. В качестве примера см. Руководство по комплексу средств защиты ОС Альт 8 СП, п. 3.6.5. «Пример настройки системы резервного копирования данных».

Управление доступом

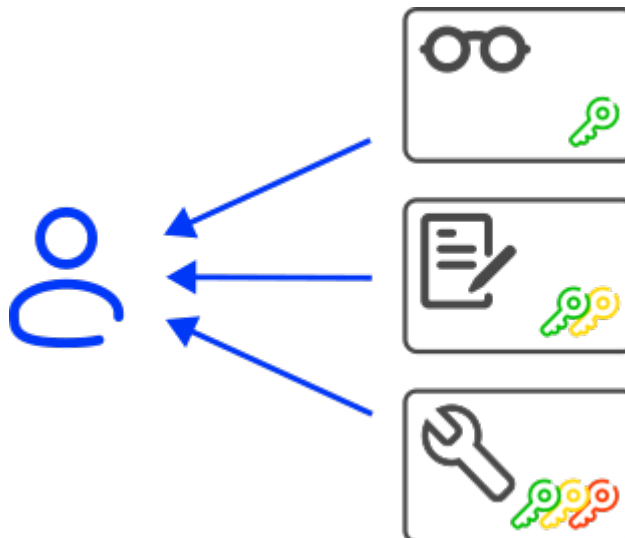
В данном разделе описана ролевая модель управления доступом в Picodata и приведены примеры необходимых для ее настройки с помощью DCL-команд языка SQL.

Ролевая модель

Ролевая модель Picodata позволяет гибко разграничивать возможности пользователей распределенной системы. Разграничение строится на базе трех основных понятий: пользователей, привилегий и ролей. Привилегии можно назначать напрямую пользователям:



Также, привилегии могут быть назначены роли, которая, в свою очередь, может быть присвоена пользователю:



У каждого пользователя может быть одна или несколько ролей. Каждому пользователю или роли может быть назначена одна или несколько привилегий. При этом, сразу после создания новая роль не содержит привилегий: их нужно явно ей присвоить.

Пользователь, который назначает привилегию, должен сам обладать ею. Глобальные привилегии может назначать только администратор СУБД.

Объекты доступа

Picodata является распределенной СУБД, и управление доступом происходит централизованно на всем кластере. Для управления доступом в Picodata используются дискреционный и ролевой методы. Объектами доступа являются:

- table – таблица БД
- user – учетная запись пользователя СУБД
- role – роль
- procedure – процедура

Доступ к объектам предоставляется на основе настраиваемого списка управления доступом (access control list, ACL), который определяет, какими привилегиями обладает пользователь или роль.

Пользователи СУБД

При подключении к системе пользователь указывает имя учетной записи. Действия, которые пользователь может совершать в системе, определяются выданными ему привилегиями.

Каждый объект в системе (таблица, роль, учетная запись, процедура) имеет привязанного к нему владельца — пользователя СУБД. Владелец объекта является его создатель. Он обладает неотъемлемыми полными привилегиями на этот объект.

Системные пользователи

Picodata предоставляет несколько встроенных учетных записей для системных пользователей:

- admin — Администратор СУБД
- pico_service — служебный пользователь, от имени которого происходит коммуникация между инстансами кластера
- guest — неавторизованный пользователь. У этого пользователя по умолчанию отсутствует пароль и привилегия LOGIN, поэтому подключиться от его имени через `psql` невозможно

Примечание

Для установки пароля для служебного пользователя `pico_service` поместите в рабочую директорию инстанса файл `.picodata-cookie`, содержащий пароль. Это нужно сделать до первого запуска инстанса.

Администратор СУБД

Администратором СУБД является встроенный пользователь `admin`.

Администратор СУБД является *суперпользователем* и обладает следующими привилегиями:

- создавать учетные записи пользователей СУБД
- модифицировать, блокировать и удалять учетные записи пользователей СУБД

- назначать права доступа пользователям СУБД к объектам доступа СУБД
- управлять конфигурацией СУБД
- создавать, подключать БД

Администратор БД

Наделить пользователя `alice` правами Администратора БД можно следующим набором SQL-команд:

```
CREATE ROLE db_admin;
GRANT READ TABLE TO db_admin;
GRANT CREATE TABLE TO db_admin;
GRANT CREATE USER TO db_admin;
GRANT CREATE ROLE TO db_admin;
GRANT CREATE PROCEDURE TO db_admin;
GRANT db_admin to alice;
```

Это обеспечивает наличие у администратора БД следующих прав:

- создавать учетные записи пользователей БД
- модифицировать, блокировать и удалять учетные записи пользователей БД
- управлять конфигурацией БД через создание и удаление таблиц БД
- назначать права доступа пользователям БД к таблицам БД
- создавать резервные копии БД и восстанавливать БД из резервной копии
- создавать, модифицировать и удалять хранимые процедуры

При создании объекта пользователь становится его владельцем и автоматически получает на него неотъемлемые полные привилегии.

Пользователь БД

Только что созданный пользователь обладает:

- ролью `public`
- привилегией `login` (подключение к кластеру)

Picodata позволяет наделить пользователя БД следующими правами:

- создавать и манипулировать записями в таблицах БД
- выполнять хранимые процедуры

Для этого используйте следующие SQL-команды:

```
Пример для таблицы `warehouse` и пользователя `alice` GRANT WRITE ON TABLE warehouse TO alice;
GRANT READ ON TABLE warehouse TO alice;
```

```
Пример для процедуры `proc` и пользователя `alice` GRANT EXECUTE ON PROCEDURE proc TO alice;
```

Привилегии

В Picodata определены следующие виды привилегий:

- привилегии для работы с пользователями (`CREATE USER`, `ALTER USER`, `DROP USER`)
- привилегии для работы с ролями (`CREATE ROLE`, `DROP ROLE`)

- привилегии для работы с таблицами (CREATE TABLE, ALTER TABLE, DROP TABLE, READ TABLE, WRITE TABLE)
- привилегии для работы с процедурами (CREATE PROCEDURE, DROP PROCEDURE, EXECUTE PROCEDURE)
- LOGIN — право подключаться к экземпляру кластера. Автоматически выдается новым пользователям при создании.

Примечание

Привилегии на управление плагинами имеются только у Администратора СУБД.

Таблица привилегий

В таблице ниже показаны привилегии и их возможности по отношению к объектам доступа. Привилегии показаны отдельно для объектов, созданных текущим пользователем (*свои объекты*) и объектов, владельцем которых является кто-то другой (*чужие объекты*).

Легенда

SQL-команды выделены в таблице цветом, в зависимости от их типа:

- Data Definition Language (DDL)
- Data Control Language (DCL)
- Data Modification Language (DML)
- Data Query Language (DQL)

SQL-команда	Требуемая привилегия для своих объектов	Требуемая привилегия для чужих объектов
ALTER PLUGIN	доступно только для пользователя <i>admin</i>	доступно только для пользователя <i>admin</i>
ALTER PROCEDURE	не требуется	не реализовано
ALTER SYSTEM	доступно только для пользователя <i>admin</i>	доступно только для пользователя <i>admin</i>
ALTER TABLE	не требуется	ALTER TABLE
CREATE INDEX	не требуется	CREATE TABLE
CREATE PLUGIN	доступно только для пользователя <i>admin</i>	доступно только для пользователя <i>admin</i>
CREATE PROCEDURE	CREATE PROCEDURE	CREATE PROCEDURE
CREATE TABLE	CREATE TABLE	не применимо
DROP INDEX	• Своя таблица + свой индекс: не требуется • Чужая таблица + свой индекс: DROP TABLE	• Своя таблица + чужой индекс: не требуется • Чужая таблица + чужой индекс: DROP TABLE
DROP PLUGIN	доступно только для пользователя <i>admin</i>	доступно только для пользователя <i>admin</i>
DROP PROCEDURE	не требуется	DROP PROCEDURE
DROP TABLE	не требуется	DROP TABLE
TRUNCATE TABLE	не требуется	WRITE TABLE
ALTER USER	не требуется	ALTER USER
CREATE ROLE	не применимо	CREATE ROLE
CREATE USER	не применимо	CREATE USER

SQL-команда	Требуемая привилегия для своих объектов	Требуемая привилегия для чужих объектов
DROP ROLE	не требуется	DROP ROLE
DROP USER	не требуется	DROP USER
GRANT	не требуется	доступно только для пользователя <i>admin</i>
REVOKE	не требуется	доступно только для пользователя <i>admin</i>
AUDIT POLICY	доступно только для пользователя <i>admin</i>	доступно только для пользователя <i>admin</i>
CALL	- Своя таблица + своя процедура: не требуется - Чужая таблица + своя процедура: WRITE TABLE	- Своя таблица + чужая процедура: EXECUTE PROCEDURE - Чужая таблица + чужая процедура: EXECUTE PROCEDURE + WRITE TABLE
DELETE	не требуется	READ TABLE + WRITE TABLE
INSERT	не требуется	WRITE TABLE
UPDATE	не требуется	READ TABLE + WRITE TABLE
SELECT	не требуется	READ TABLE

Роли

Роль представляет собой именованную группу привилегий, что позволяет структурировать управление доступом. Чтобы выдать или отозвать привилегию у роли, используйте команды GRANT и REVOKE.

Названия ролей `public`, `super` и `replication` являются зарезервированными.

Роль `public` автоматически назначается всем создаваемым пользователям. Наделение роли `public` привилегией на какой-либо объект делает его общедоступным.

Роль `super` является системной и используется исключительно для внутренних целей.

Роль `replication` зарезервирована как системная для корректной работы механизма репликации.

Начало работы

Для начала работы с пользователями, привилегиями и ролями следует подключиться к инстансу Picodata.

Управление пользователями

Методы аутентификации

При создании и модификации пользователей в Picodata поддерживаются следующие методы аутентификации:

- `md5` (используется по умолчанию)
- `ldap` (предназначен для для доступа доменных учетных записей)
- `chap-sha1` (рекомендуется использовать только для системных пользователей)
- `scram-sha256` (безопасный метод аутентификации, сохраняющий пароль в зашифрованном виде на сервере. Предназначен для доступа по протоколу PostgreSQL)

- cert (аутентификация по клиентскому сертификату, см. Аутентификация по сертификату)

Для методов md5, chap-sha1 и scram-sha256 действуют требования. Для методов ldap и cert пароль не требуется и игнорируется.

Примечание

При подключении по протоколу PostgreSQL (например, через psql) поддерживаются только методы md5, scram-sha256, ldap и cert. Метод chap-sha1 **не поддерживается** для этого протокола. По умолчанию новые пользователи создаются с методом md5.

Создание

Для создания пользователя используйте SQL-команду CREATE USER.

Пример:

```
CREATE USER alice WITH PASSWORD 'P@ssw0rd' USING md5;  
CREATE USER bob USING ldap;
```

Требования к паролю

Для методов аутентификации md5, chap-sha1 и scram-sha256 пароль должен соответствовать следующим требованиям:

- не менее 8 символов
- минимум одна заглавная буква
- минимум одна строчная буква
- минимум одна цифра

Подробнее см. ниже.

Для имени пользователя (и в целом для объектов в Picodata) действуют правила использования допустимых символов. Также следует учитывать требования к длине и сложности пароля.

Для выполнения команды требуется привилегия CREATE USER:

```
GRANT CREATE USER TO alice;
```

Такое право по умолчанию есть у Администратора СУБД (admin).

Модификация

Для изменения учетных данных пользователя используйте SQL-команду ALTER USER.

Для выполнения команды требуется привилегия ALTER USER на конкретную учетную запись или на все учетные записи сразу:

```
GRANT ALTER ON USER bob TO alice;  
GRANT ALTER USER TO alice;
```

Привилегия ALTER USER есть по умолчанию у создателя данной учетной записи и у Администратора СУБД.

Блокирование

Для блокировки пользователя `alice` используйте следующую SQL-команду:

```
ALTER USER alice WITH NOLOGIN;
```

После 4 неуспешных попыток аутентификации пользователь блокируется автоматически.

Для разблокировки пользователя используйте следующую SQL-команду:

```
ALTER USER alice WITH LOGIN;
```

Для выполнения команд требуется привилегия `ALTER USER` — на все учетные записи или на конкретную.

По умолчанию привилегия `ALTER USER` есть у создателя учетной записи и у администратора СУБД.

Удаление

Для удаления пользователя `alice` используйте SQL-команду `DROP USER`:

```
DROP USER alice;
```

Для выполнения команды требуется привилегия `DROP USER` на конкретную учетную запись или на все учетные записи сразу:

```
GRANT DROP ON USER bob TO alice;
GRANT DROP USER TO alice;
```

Привилегия `DROP USER` есть по умолчанию у создателя данной учетной записи и у Администратора СУБД.

Просмотр списка пользователей

Пользователи Picodata хранятся в глобальной системной таблице `_pico_user`. Для просмотра всего списка используется следующая команда:

```
SELECT * FROM _pico_user;
```

Доступ к списку пользователей есть у Администратора СУБД, а также у тех пользователей, которым была явно выдана такая привилегия:

```
GRANT READ ON TABLE _pico_user TO alice;
```

Установка и изменение пароля

Для установки или изменения пароля на `T0psecret` для пользователя `alice` используйте следующую SQL-команду:

```
ALTER USER "alice" WITH PASSWORD 'T0psecret';
```

Требования к паролю

При установке или изменении пароля пользователя следует учитывать требования к его длине и сложности в том случае, если используются методы аутентификации `md5`, `chap-sha1` или `scram-sha256` (для метода `ldap` пароль не требуется и игнорируется):

- пароль должен быть не короче 8 символов

- пароль должен одновременно содержать минимум один символ в нижнем регистре, символ в верхнем регистре и одну цифру

Пример некорректного пароля: topsecret

Пример корректного пароля: T0psecret

Использование ролей

Для создания и удаления ролей используйте команды CREATE ROLE и DROP ROLE.

Выполнение данных действий требует наличия привилегий CREATE ROLE / DROP ROLE соответственно.

Для назначения роли manager пользователю alice используйте команду GRANT:

```
GRANT manager TO alice;
```

Назначение привилегии ALTER на таблицу warehouse для роли manager:

```
На конкретную таблицу `warehouse` GRANT WRITE ON TABLE warehouse TO manager;
```

```
На все таблицы GRANT WRITE TABLE TO manager;
```

В подобных командах можно использовать как идентификатор роли, так и пользователя. Стоит отметить, что не все привилегии можно выдать ролям, например, привилегия SESSION не может быть выдана другой роли при помощи GRANT, а только командой ALTER USER.

Выдать или отозвать роль может только ее создатель. Исключением является администратор СУБД, который может выдать или отозвать любую роль.

Отозвать роль можно с помощью команды REVOKE:

```
Отзыв привилегии у роли REVOKE WRITE ON TABLE warehouse FROM manager;
```

```
Отзыв роли у пользователя REVOKE manager FROM alice;
```

Управление доступом к веб-интерфейсу

По умолчанию, для доступа к веб-интерфейсу необходимо пройти аутентификацию, указав действующее имя учетной записи в СУБД Picodata и корректный пароль. Аутентификация доступна всем пользователям с привилегией LOGIN.

Администратор БД может явным образом включать/выключать режим аутентификации. Когда аутентификация отключена, доступ к веб-интерфейсу не ограничивается — страница с обзором кластера открывается сразу.

Режим аутентификации для веб-интерфейса управляется параметром jwt_secret в системной таблице _pico_db_config. Для отключения аутентификации задайте параметру пустое значение:

```
ALTER SYSTEM SET jwt_secret = '';
```

Для того, чтобы включить аутентификацию обратно, сбросьте значение параметра:

```
ALTER SYSTEM RESET jwt_secret;
```

Внимание!

Для повышения безопасности необходимо использовать SSL-терминатор (обратный прокси с SSL-терминацией) — промежуточный сервер, который принимает зашифрованный HTTPS-трафик от клиентов, расшифровывает его и перенаправляет на внутренние серверы в виде обычного HTTP-запроса. Это позволит не передавать по сети пароль пользователя в открытом виде.

Управление доступом к таблицам

Picodata позволяет задавать разрешение пользователям и процедурам выполнять следующие операции в отношении таблиц БД: создание, модификация, удаление, чтение.

Для наделения пользователя `alice` и созданных им процедур указанными привилегиями на таблицу `warehouse` используйте SQL-команды GRANT:

```
GRANT CREATE TABLE TO alice;
GRANT ALTER ON TABLE warehouse TO alice;
GRANT DROP ON TABLE warehouse TO alice;
GRANT READ ON TABLE warehouse TO alice;
GRANT WRITE ON TABLE warehouse TO alice;
```

Отозвать привилегию можно SQL-командой REVOKE. Например:

```
REVOKE DROP ON TABLE warehouse FROM alice;
```

В качестве `<table_name>` можно использовать имена системных таблиц — по умолчанию доступ к ним запрещен всем пользователям СУБД, кроме `admin` и `pico_service`.

Привилегию на создание таблиц можно использовать как на все таблицы (CREATE TABLE), так и на явно указанную таблицу (CREATE ON TABLE `<table_name>`):

```
GRANT CREATE ON TABLE sklad TO alice;
```

Соответственно, администратор может отозвать конкретную привилегию через REVOKE:

```
REVOKE CREATE ON TABLE sklad FROM alice;
```

Управление доступом к процедурам

Picodata позволяет задавать разрешение пользователям СУБД выполнять следующие операции в отношении процедур: создание, модификация, удаление, исполнение.

Для наделения пользователя `alice` указанными привилегиями для процедуры `proc` используйте SQL-команду GRANT:

```
GRANT CREATE PROCEDURE TO alice;
GRANT DROP ON PROCEDURE proc TO alice;
GRANT EXECUTE ON PROCEDURE proc TO alice;
```

Отозвать привилегию можно SQL-командой REVOKE. Например:

```
REVOKE DROP ON PROCEDURE proc FROM alice;
```

Аутентификация с помощью LDAP

В данном разделе приведены сведения об аутентификации в Picodata с помощью протокола LDAP, в том числе с поддержкой TLS.

Picodata Enterprise

Данная функция доступна только в коммерческой версии Picodata.

Общие сведения

Lightweight Directory Access Protocol (далее LDAP) — это легковесный протокол для доступа к службе каталогов X.500, которая часто используется организациями для контроля доступа к ресурсам и управления доменными учетными записями.

Существует множество серверов, которые поддерживают данный протокол: OpenLDAP, GLAuth и прочие.

Механизм аутентификации пользователей

Типичный сеанс связи с сервером LDAP выглядит примерно так: клиент подключается к серверу, отправляет запрос, дожидается ответа сервера и закрывает соединение. На основании ответа сервера клиент может принять решение об аутентификации пользователя, инициировавшего подключение.

В качестве запроса в Picodata используется BIND, позволяющий произвести аутентификацию пользователя с использованием заранее известного Distinguished Name (DN) и факторов аутентификации (например, пароля).

Для использования протокола LDAP в Picodata следует задать для учетной записи пользователя тип аутентификации ldap. Примеры см. ниже.

Особенности работы LDAP

- Коммуникация с сервером LDAP происходит один раз при установке новой пользовательской сессии. Если аутентификация пройдена успешно, сессия будет установлена, и до самого её конца повторная аутентификация производиться не будет.
- **Метод отвечает только за аутентификацию.** Для синхронизации групп, прав и владения объектами с удаленным сервером LDAP следует использовать плагин Argus.
- Метод требует дополнительной настройки кластера через файл конфигурации Picodata.
- Метод невозможно использовать в случае отказа удаленного сервера LDAP.
- Метод не кеширует результаты аутентификации.

Поддержка защищённого режима

Зашифрованная коммуникация по протоколу LDAP возможна в двух вариантах:

- LDAP over TLS (implicit): подключение выполняется к отдельному порту (636), и полностью инкапсулируется внутри TLS/SSL. В этом случае никакой незашифрованный

LDAP-трафик по каналу не проходит.

- LDAP + StartTLS (`start_tls`): изначальное подключение выполняется к обычному порту (389) и начинается в незашифрованном виде. Потом клиент посылает серверу команду StartTLS и подключение начинает использовать шифрование.

Оба варианта обеспечивают конфиденциальность и безопасность данных при доступе к службам каталогов. Если сервер слушает отдельный порт для LDAPS (обычно 636), необходимо использовать метод `implicit`; если шифрование настроено поверх стандартного LDAP-порта (389) — `start_tls`.

Для включения любого из этих режимов необходимо выставить параметр `instance.ldap.tls.enabled` в значение `true`.

За выбор метода использования TLS отвечает параметр `instance.ldap.tls.method`: `implicit` использует LDAP over TLS (LDAPS), а `start_tls` — LDAP + StartTLS.

После выполнения настроек зашифрованное подключение будет работать автоматически если на стороне LDAP-сервера используется сертификат, подписанный корневым сертификатом (CA), которому доверяет клиент.

В случае если такой корневой сертификат не находится в системном хранилище доверенных сертификатов, возможно использование опции `instance.ldap.tls.ca_file` для указания альтернативных доверенных корневых сертификатов.

Настройка

Для использования LDAP-аутентификации под определенным пользователем в Picodata требуется:

- наличие в сети запущенного сервера LDAP, на котором заведен этот пользователь
- установленные настройки узлов Picodata
- наличие пользователя с таким же именем и типом аутентификации `ldap` в Picodata

Настройка инстанса для аутентификации LDAP

Для корректной работы метода нужно установить следующие параметры Picodata на каждом инстансе, который должен принимать LDAP-аутентификацию:

- `instance.ldap.enabled` — признак поддержки LDAP-аутентификации на инстансе. Необходимое значение — `true`.
- `instance.ldap.connect` — строка `<host>:<port>`, указывающая на удаленный сервер LDAP.
- `instance.ldap.dn_format` — формат-строка DN для запроса BIND.
- `instance.ldap.tls.enabled` — включение/выключение TLS.
- `instance.ldap.tls.method` — метод TLS-подключения.
- `instance.ldap.tls.ca_file` — путь к файлу с альтернативными доверенными корневыми сертификатами (опционально).

Пример конфигурационного файла инстанса `i1`, подключающегося к `127.0.0.1:1389` с использованием LDAP over TLS:

`i1.yml`

```
cluster:
  name: my_cluster
  tier:
    default:

instance:
  instance_dir: './i1'
  name: 'i1'
  tier: 'default'
  peer: [ 127.0.0.1:3301 ]

iproto:
  listen: '0.0.0.0:3301'
  advertise: '127.0.0.1:3301'
http:
  listen: '0.0.0.0:8081'
pgproto:
  listen: '0.0.0.0:4327'
  advertise: '127.0.0.1:4327'
ldap:
  enabled: true
  connect: '127.0.0.1:1636'
  dn_format: 'cn=$USER,ou=users,dc=example,dc=org'
  tls:
    enabled: true
    method: 'implicit'
    # если необходимо использовать Root CA не из системного хранилища
    # ca_file: '/etc/picodata/ldap-ca.pem'
```

Здесь 1636 используется как нестандартный порт, типичный для тестовых LDAP-серверов (например, GLAuth). В продуктовой среде обычно используется стандартный порт 636 для LDAPS либо 389 для LDAP/StartTLS.

Для того чтобы такая конфигурация работала, имя сервера, указанное в `instance.ldap.connect`, должно совпадать с именем, указанным в сертификате LDAP-сервера (SAN или CN), а также этот сертификат должен быть доверенным.

В текущей реализации Picodata строка форматирования DN общая для всех пользователей. Кроме того, нет возможности установить персональный DN для конкретного пользователя. В строке `dn_format` доступна только подстановка имени пользователя через `$USER`.

Запуск кластера, состоящего из одного инстанса `i1`, с аутентификацией в LDAP:

```
picodata run --config i1.yml
```

Настройка пользователей LDAP в Picodata

Как и другие методы аутентификации, `ldap` можно указать при создании нового пользователя через параметр `USING`:

```
CREATE USER "username" USING ldap;
```

Помимо этого, можно сменить метод аутентификации уже существующего пользователя:

```
ALTER USER "username" USING ldap;
```

Обратите внимание, что при использовании метода LDAP **не требуется** указывать пароль пользователя. Это связано с тем, что Picodata не хранит пароли: при каждой попытке аутентификации клиент будет отправлять пароль отдельно; в свою очередь, Picodata будет использовать полученный пароль при взаимодействии с сервером LDAP. Также рекомендуется включить TLS для передачи пароля по сети в зашифрованном виде.

Устранение неполадок

После запуска инстанса и настройки пользователя попробуйте подключиться под созданным пользователем через `psql`; в случае ошибки проверьте журналы инстанса.

В отличие от других методов аутентификации, LDAP требует дополнительной настройки, без которой аутентификация с использованием этого метода будет невозможна. Также важно понимать, что любые проблемы с удаленным сервером LDAP немедленно отразятся на возможности аутентифицировать пользователей. Для решения этой проблемы администраторам предлагается временно изменить метод аутентификации (например, на `'md5'`) для затронутых пользователей при помощи функции смены пароля:

```
ALTER USER "username" WITH PASSWORD 'new_password' USING md5;
```

LDAP BIND-запросы регистрируются в журнале на уровне `verbose`. Ошибки взаимодействия регистрируются на уровне `warn`, а также сообщаются клиенту, подключающемуся по PostgreSQL-протоколу.

Подключение к кластеру в Oracle Weblogic

В данном разделе приведены сведения о настройке подключения к работающему кластеру Picodata в Oracle Weblogic Server — коммерческом сервере Java-приложений.

Общие сведения

Для использования Java-приложений с СУБД Picodata необходим JDBC-драйвер Picodata. Драйвер импортируется в Weblogic Server и позволяет использовать СУБД Picodata как источник данных (Data Source). Поскольку кластер Picodata — распределённая система, то для каждого репликасета кластера следует создать по отдельному источнику данных и затем объединить их в общий мульти-источник для того, что обеспечить балансировку нагрузки при работе с данными. В дальнейшем, при развёртывании клиентских приложений в Weblogic Server, следует использовать именно этот мульти-источник.

Примечание

В данном руководстве для примера приводится Oracle Weblogic Server 12, работающий под Oracle Java 8. Настройки для других версий могут отличаться.

Предварительные требования

Для выполнения действий, описанных в данном руководстве, понадобятся:

- доступ к работающему кластеру Picodata, в том числе, возможность повышения привилегий до Администратора СУБД
- драйвер picodata-jdbc
- установленный и развёрнутый Oracle Weblogic Server, включая доступ у его консоли управления

Подготовка окружения

Выполните следующие шаги:

- разверните Oracle Weblogic Server согласно документации производителя и настройте сервер (например, запустив исполняемый файл `Oracle_Home/oracle_common/common/bin/config.sh`)
- загрузите дистрибутив драйвера picodata-jdbc с включёнными зависимостями (`picodata-jdbc-x.x.x-shaded.jar`) и положите его в директорию `DOMAIN_HOME/lib`
- запустите Weblogic Server с помощью исполняемого файла `startWebLogic.sh` в директории `DOMAIN_HOME/bin`
- запустите кластер Picodata, определив для каждого инстанса порт приёма подключений по протоколу PostgreSQL. Указанные порты будут использоваться в дальнейшем JDBC-драйвером Picodata
- в Picodata создайте пользователя, под которым JDBC-драйвер сможет авторизоваться в кластере. Важно указать md5 в качестве типа авторизации. Убедитесь, что подключение к кластеру по протоколу PostgreSQL с выбранными параметрами

корректно работает.

Настройка источников данных

Добавление источника

Авторизуйтесь в консоли управления Oracle Weblogic Server

(<http://<server>:7001/console>) и перейдите в раздел **Services > Data Sources**:

ORACLE WebLogic Server Administration Console 12c

Home Log Out Preferences Record Help Welcome, weblogic Connected to: base_domain

Home > Summary of JDBC Data Sources

Summary of JDBC Data Sources

Configuration Monitoring

A JDBC data source is an object bound to the JNDI tree that provides database connectivity through a pool of JDBC connections. Applications can look up a data source on the JNDI tree and then borrow a database connection from a data source.

This page summarizes the JDBC data source objects that have been created in this domain.

[Customize this table](#)

Data Sources (Filtered - More Columns Exist)

New Delete Showing 0 to 0 of 0 Previous Next

☐	Name	Type	JNDI Name	Targets
There are no items to display				

New Delete Showing 0 to 0 of 0 Previous Next

How do I...

- Create JDBC generic data sources
- Create JDBC GridLink data sources
- Create JDBC multi data sources
- Create UCP data sources
- Create Proxy data sources

System Status

Health of Running Servers as of 18:00

- Failed (0)
- Critical (0)
- Overloaded (0)
- Warning (0)
- OK (1)

Создайте новый обычный источник данных (**New > Generic Data Source**). Заполните следующие поля:

- имя источника (*Name*)
- имя JNDI (*JNDI Name*), по которому клиентские приложения смогут идентифицировать источник данных
- тип базы данных (*Database Type*) — выберите вариант **Other**

ORACLE WebLogic Server Administration Console 12c

Home Log Out Preferences Record Help Welcome, weblogic Connected to: base_domain

Home > Summary of JDBC Data Sources

Create a New JDBC Data Source

Back Next Finish Cancel

JDBC Data Source Properties

The following properties will be used to identify your new JDBC data source.

* Indicates required fields

What would you like to name your new JDBC data source?

Name: Picodata JDBC-0

What scope do you want to create your data source in ?

Scope: Global

What JNDI name would you like to assign to your new JDBC Data Source?

JNDI Name: picodata-jdbc-source-0

What database type would you like to select?

Database Type: Other

Back Next Finish Cancel

Нажмите **Next** для подтверждения настроек, и затем ещё раз **Next**. На следующем экране снимите флаг с пункта, отвечающего за поддержку глобальных транзакций (**Supports Global Transactions**):

ORACLE WebLogic Server Administration Console 12c

Home Log Out Preferences Record Help Welcome, weblogic Connected to: base_domain

Home > Summary of JDBC Data Sources

Create a New JDBC Data Source

Back Next Finish Cancel

Transaction Options

You have selected non-XA JDBC driver to create database connection in your new data source.

Does this data source support global transactions? If yes, please choose the transaction protocol for this data source.

☐ **Supports Global Transactions**

Select this option if you want to enable non-XA JDBC connections from the data source to participate in global transactions using the *Logging Last Resource* (LLR) transaction optimization. Recommended in place of Emulate Two-Phase Commit.

☐ **Logging Last Resource**

Select this option if you want to enable non-XA JDBC connections from the data source to emulate participation in global transactions using JTA. Select this option only if your application can tolerate heuristic conditions.

☐ **Emulate Two-Phase Commit**

Select this option if you want to enable non-XA JDBC connections from the data source to participate in global transactions using the one-phase commit transaction processing. With this option, no other resources can participate in the global transaction.

☐ **One-Phase Commit**

Back Next Finish Cancel

Нажмите **Next** для перехода к следующему шагу. Укажите имя пользователя и пароль для подключения к базе данных (пользователь с этими данными должен существовать в Picodata).

ORACLE WebLogic Server Administration Console 12c

Home Log Out Preferences Record Help Welcome, weblogic Connected to: base_domain

Home > Summary of JDBC Data Sources

Create a New JDBC Data Source

Back Next Finish Cancel

Connection Properties

Define Connection Properties.

What database account user name do you want to use to create database connections?

Database User Name: sqluser

What is the database account password to use to create database connections?

Password:

Confirm Password:

Back Next Finish Cancel

Change Center

View changes and restarts

Configuration editing is enabled. Future changes will automatically be activated as you modify, add or delete items in this domain.

Domain Structure

- base_domain
 - Domain Partitions
 - Environment
 - Deployments
 - Services
 - Messaging
 - Data Sources
 - Persistent Stores
 - Foreign JNDI Providers
 - Work Contexts
 - XML Registries
 - XML Entity Caches
 - jCOM
 - Mail Sessions

How do I...

- Create JDBC generic data sources
- Create LLR-enabled JDBC data sources

System Status

Health of Running Servers as of 18:14

Failed (0)
Critical (0)
Overloaded (0)
Warning (0)
OK (1)

Нажмите **Next** для перехода к следующему шагу. Укажите данные JDBC-драйвера и параметры подключения:

- Driver Class Name: `io.picodata.jdbc.Driver`
- URL: `jdbc:picodata://localhost:5432/`
- Database User Name: `sqluser` (пример пользователя, под которым драйвер сможет авторизоваться в Picodata)
- поля Password: и Confirm Password: — идентичные экземпляры пароля для указанного выше пользователя

ORACLE WebLogic Server Administration Console 12c

Home Log Out Preferences Record Help Welcome, weblogic Connected to: base_domain

Change Center
View changes and restarts
Configuration editing is enabled. Future changes will automatically be activated as you modify, add or delete items in this domain.

Domain Structure
base_domain
- Domain Partitions
- Environment
- Deployments
- Services
- Messaging
- Data Sources
- Persistent Stores
- Foreign JNDI Providers
- Work Contexts
- XML Registries
- XML Entity Caches
- JCOM
- Mail Sessions

How do I...
• Create JDBC generic data sources
• Create LLR-enabled JDBC data sources

System Status
Health of Running Servers as of 18:17
Failed (0)
Critical (0)
Overloaded (0)
Warning (0)
OK (1)

Create a New JDBC Data Source
Test Configuration Back Next Finish Cancel

Test Database Connection
Test the database availability and the connection properties you provided.

What is the full package name of JDBC driver class used to create database connections in the connection pool?
(Note that this driver class must be in the classpath of any server to which it is deployed.)

Driver Class Name: io.picodata.jdbc.Driver

What is the URL of the database to connect to? The format of the URL varies by JDBC driver.

URL: jdbc:picodata://localhost:5432/

What database account user name do you want to use to create database connections?

Database User Name: sqluser

What is the database account password to use to create database connections?
(Note: for secure password management, enter the password in the Password field instead of the Properties field below)

Password: *****

Confirm Password: *****

What are the properties to pass to the JDBC driver when creating database connections?

Properties: user=sqluser

В качестве значения URL можно указать несколько экземпляров, относящихся к одному и тому же репликасету в кластере Picodata, например:

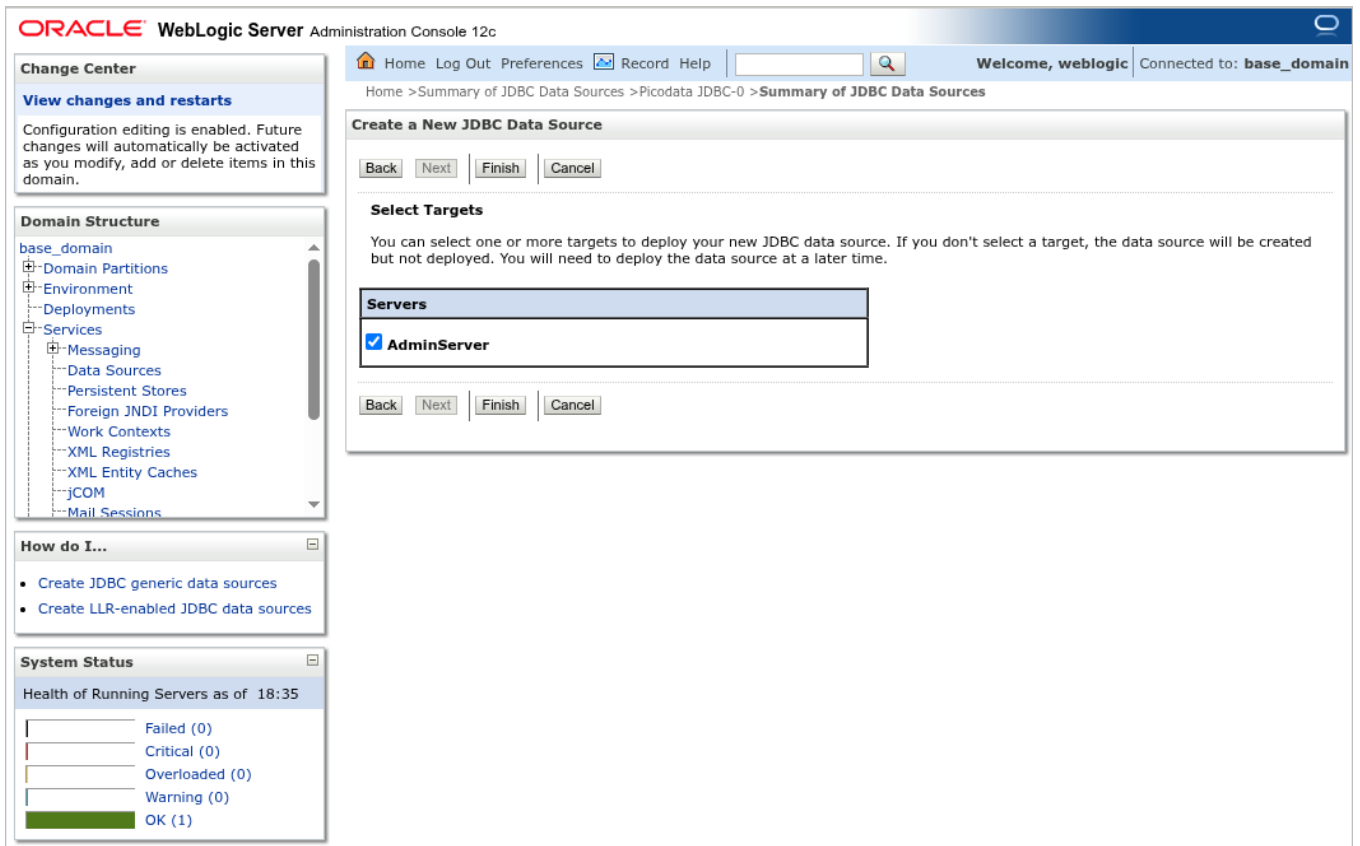
```
jdbc:picodata://srv1:5432,srv2:5432/
```

Примечание

В примере использован порт 5432 — он должен соответствовать порту приёма подключений по протоколу PostgreSQL для каждого экземпляра. Также, в конце URL должен обязательно стоять закрывающий знак /.

Проверьте корректность настроек с помощью кнопки **Test Configuration**: должно появиться сообщение *Connection test succeeded*. Если это не так, то при вводе данных была сделана ошибка, либо экземпляр Picodata по какой-либо причине не принимает подключения (сетевая недоступность, пользователь с неверным типом авторизации и т.д.).

Нажмите **Next** для перехода к следующему шагу. Отметьте целевой сервер, на котором будет развернут источник данных:



Нажмите **Finish**. Добавление и развёртывание источника данных завершено.

Аналогичным образом добавьте источники данных для остальных инстансов кластера.

Объединение источников

Для объединения ранее созданных источников данных перейдите в раздел **Services > Data Sources** и создайте новый мульти-источник (**New > Multi Data Source**).

Заполните следующие поля:

- имя мульти-источника (*Name*)
- имя JNDI (*JNDI Name*), по которому клиентские приложения смогут идентифицировать мульти-источник данных
- тип алгоритма (*Algorithm Type*), по которому будут использоваться разные вложенные источники — выберите вариант **Load Balancing** (алгоритм балансировки по принципу Round Robin).

ORACLE WebLogic Server Administration Console 12c

Home Log Out Preferences Record Help Welcome, weblogic Connected to: base_domain

Home > Summary of JDBC Data Sources > Picodata JDBC-0 > Summary of JDBC Data Sources

Create a New JDBC Multi Data Source

Back Next Finish Cancel

Configure the Multi Data Source

The following properties will be used to identify your new JDBC multi data source.
* Indicates required fields

What would you like to name your new JDBC multi data source?

* **Name:** Picodata Multi Source

What scope do you want to create your JDBC Multi Data Source in ?

Scope: Global

What JNDI name would you like to assign to your new JDBC multi data source?

* **JNDI Name:** picodata-multi-source

What algorithm type for this JDBC Multi Data Source would you like to select?

* **Algorithm Type:** Load-Balancing

Back Next Finish Cancel

Нажмите **Next** для перехода к следующему шагу. Отметьте целевой сервер, на котором будет развёрнут источник данных и нажмите **Next**.

Убедитесь, что на данном шаге выбран вариант **Non-XA Driver** — именно к нему относятся ранее добавленные обычные источники данных.

ORACLE WebLogic Server Administration Console 12c

Home Log Out Preferences Record Help Welcome, weblogic Connected to: base_domain

Home > Summary of JDBC Data Sources > Picodata JDBC-0 > Summary of JDBC Data Sources

Create a New JDBC Multi Data Source

Back Next Finish Cancel

Select Data Source Type

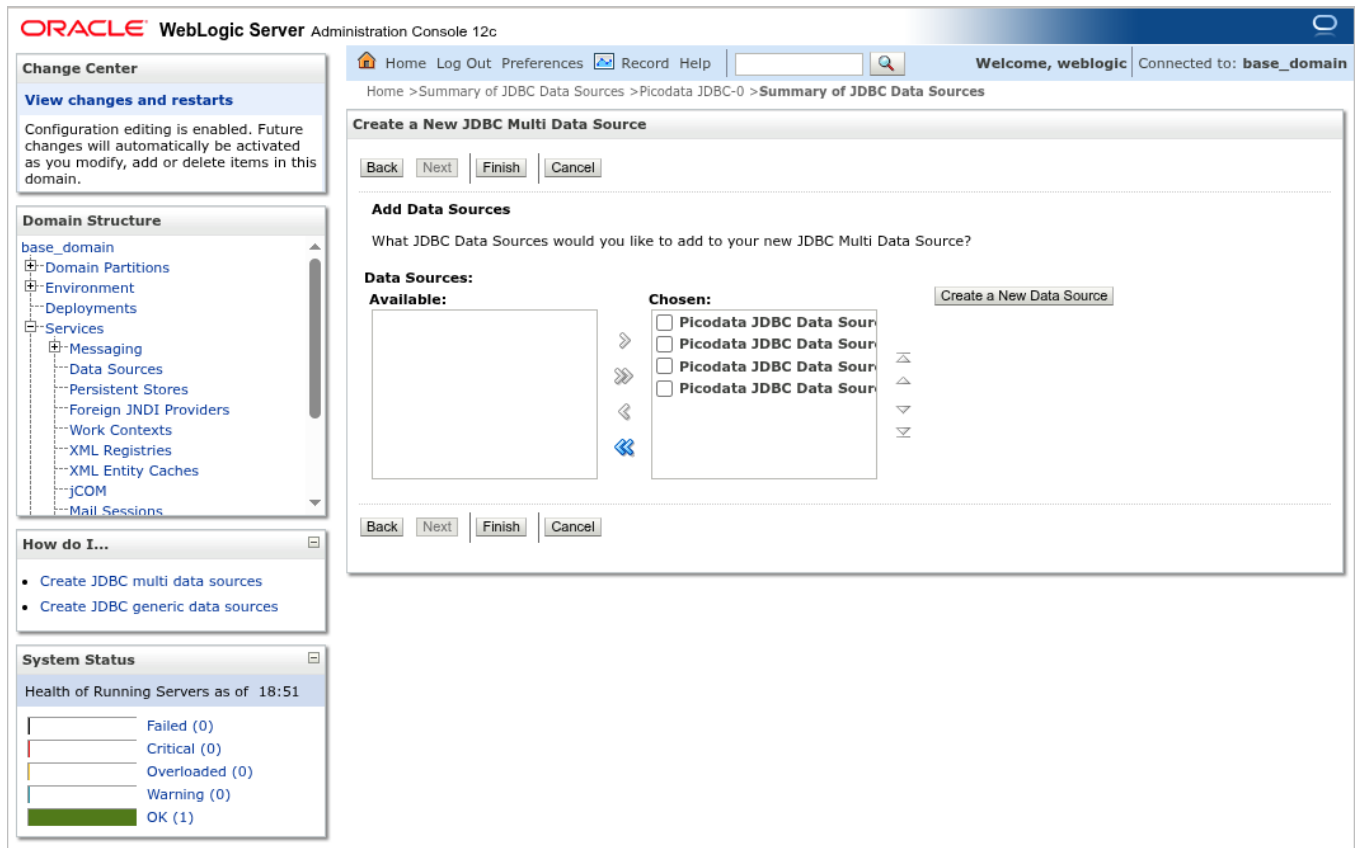
Please select type (XA or Non-XA) of data source you would like to add to your new JDBC Multi Data Source.

☐ XA Driver

☒ **Non-XA Driver**

Back Next Finish Cancel

Нажмите **Next** для перехода к следующему шагу. Добавьте в состав мульти-источника ранее созданные обычные источники.



Нажмите **Finish**. Добавление и развёртывание мульти-источника данных завершено.

Безопасность кластера

Подключение по протоколу PostgreSQL в безопасном режиме

Picodata поддерживает TLS/SSL при подключении по протоколу PostgreSQL, которое обеспечивает безопасную коммуникацию между клиентом (окном терминала или клиентским API) и сервером (инстансом Picodata).

В частности, при проверке подлинности с помощью пароля, использование TLS/SSL позволяет не передавать его по сети в открытом виде.

Безопасное соединение задействуется автоматически если оно поддерживается (и настроено) на стороне инстанса Picodata.

Примечание

Если в конфигурации инстанса включен параметр `instance.pgproto.tls.enabled`, то небезопасные соединения (без TLS/SSL) при этом становятся запрещены.

Настройка на стороне сервера

Безопасный режим — это подключение после проверки сертификатов сервера (инстанса Picodata) и клиента (например, `psql`). Возможна настройка в двух режимах:

- односторонняя проверка сервера клиентом (режим TLS/SSL). Требуется пара сертификат + закрытый ключ на сервере (`instance.pgproto.ssl.cert_file` и `instance.pgproto.ssl.key_file`) и корневой сертификат на клиенте (`ca.crt`)
- двусторонняя проверка сервера и клиента друг другом (режим mTLS). На обеих сторонах нужны по 3 файла: `instance.pgproto.ssl.cert_file`, `instance.pgproto.ssl.key_file` и `instance.pgproto.ssl.ca_file` на сервере, и `client.crt`, `client.key` и `ca.crt` на клиенте. В результате PostgreSQL-сервер в Picodata будет принимать подключения только в том случае, если клиент предоставит сертификат, подписанный с помощью такого же `ca.crt`, у которого поле CN совпадает с именем пользователя (см. Аутентификация по сертификату).

Для включения TLS/SSL на стороне Picodata установите в файле конфигурации для параметра `instance.pgproto.tls.enabled` значение `true` и положите на сервер сертификат и ключ. Укажите пути к ним через параметры командной строки или конфигурационный файл. Пример:

```
pgproto:
  tls:
    enabled: true
    cert_file: tls/custom_server.crt
    key_file: tls/custom_server.key
```

Если нужно включить проверку клиентского сертификата (mTLS), добавьте параметр `instance.pgproto.tls.ca_file` с путем до CA-сертификата, который может верифицировать клиентские сертификаты. Пример:

```
pgproto:
  tls:
    enabled: true
    cert_file: tls/custom_server.crt
```

```
key_file: tls/custom_server.key
ca_file: tls/custom_ca.crt
```

Примечание

Помимо явного указания путей через параметры конфигурации, можно положить файлы с предопределенными именами в рабочую директорию инстанса : - SSL-сертификат: `server.crt` - ключ сертификата: `server.key` - CA-сертификат (для mTLS): `ca.crt` Рекомендуется использовать явные настройки в конфигурационном файле.

Настройка на стороне клиента

При использовании приложения `psql` в режиме TLS/SSL нужно предоставить приложению CA-сертификат, соответствующий сертификату в Picodata (параметр `instance.pgproto.tls.cert_file`). По умолчанию, `psql` будет читать его из файла `~/postgresql/root.crt`. Подробнее см. в документации `libpq-ssl`.

Примеры команд для подключения:

в формате `URIpsql postgres://admin@192.168.0.22:5001?sslmode=verify-ca`

в формате `conninfo` `psql 'host=192.168.0.22 port=5001 user=admin sslmode=verify-ca'` с указанием пути к CA-файлу `psql 'host=192.168.0.22 port=5001 user=admin sslmode=verify-ca sslrootcert=/etc/picodata.ca.crt'`

Для режима mTLS понадобятся файлы с клиентским сертификатом и ключом. Сертификат должен проходить валидацию с помощью корневого сертификата `pgproto.ssl.ca_file`, настроенного в Picodata.

Пример подключения с явным указанием всех необходимых файлов:

```
psql "user=admin host=192.168.0.22 port=5001 sslrootcert=/etc/certs/ca.crt sslcert=/etc/certs/myserver1.int.crt sslkey=/etc/certs/myserver1.int.key"
```

Аутентификация по сертификату

В режиме mTLS Picodata дополнительно проверяет, кому выдан клиентский сертификат: поле Common Name (CN) в нем, обрезанное по первому символу @, должно совпадать с именем пользователя, указанным при подключении. Например, сертификаты с CN=alice и CN=alice@example.com подходят только для пользователя alice.

Проверка выполняется после обычного обмена аутентификационными сообщениями и действует для всех методов аутентификации, поэтому несовпадение имен неотличимо от неверного пароля.

Это позволяет использовать клиентский сертификат двумя способами:

- Как второй фактор аутентификации (2FA). Пользователь создан с одним из парольных методов (`md5`, `scram-sha256`, `ldap`), поэтому для подключения нужно одновременно знать пароль и предъявить сертификат, выданный этому пользователю:

```
CREATE USER "alice" WITH PASSWORD 'P@ssw0rd' USING md5;
```

- Как единственный фактор аутентификации (1FA). Пользователь создан с методом `cert`, при котором пароль не используется вовсе, и подлинность подтверждает только клиентский сертификат: Если у подключения нет подходящего сертификата

(например, mTLS не настроен или имя пользователя не совпало), то оно будет отклонено с ошибкой `certificate authentication failed for user 'alice'`.

```
CREATE USER "alice" USING cert;
```

Создание сертификатов и ключей

Ниже показаны примеры команд для генерации самоподписанного сертификата (публичного ключа) и закрытого ключа как для сервера (Picodata), так и для клиентов, которые хотят подключиться к серверу. Для этих команд используется консольное приложение `openssl` из одноименного пакета. Серверный и клиентский сертификаты должны быть подписаны одним и тем же корневым сертификатом (CA). Сертификаты должны содержать валидные имена в поле Subject Alternative Name (SAN). В примере ниже клиент находится на узле с именем `client.int`, а Picodata — на узле с именем `server.int`.

```
Шаг 1. Создание директории для сертификатов для пользователя picousermkdir -p /home/picouser/certs && cd /home/picouser/certs

Шаг 2. Создание корневого сертификата (CA)openssl genrsa -out ca.key 2048
openssl req -x509 -new -nodes -key ca.key -sha256 -days 3650 -out ca.crt -subj "/CN=RootCA"

Шаг 3. Создание серверного закрытого ключаopenssl genrsa -out server.key 2048

Шаг 4. Создание запроса на подпись серверного сертификата (CSR)openssl req -new -key server.key -out server.csr -subj "/CN=server.int"

Шаг 5. Подпись серверного сертификата с помощью корневого сертификата (CA)openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -
days 3650 -sha256 -extfile <(echo subjectAltName="DNS:server.int")

Шаг 6. Создание клиентского закрытого ключаopenssl genrsa -out client.key 2048

Шаг 7. Создание запроса на подпись клиентского сертификата (CSR)openssl req -new -key client.key -out client.csr -subj "/CN=client.int"

Шаг 8. Подпись клиентского сертификата с помощью корневого сертификата (CA) openssl x509 -req -in client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out client.crt
-days 3650 -sha256 -extfile <(echo subjectAltName="DNS:client.int")

Шаг 9. Конвертация клиентского ключа в формат PKCS#8openssl pkcs8 -topk8 -inform PEM -outform DER -in client.key -out client.pk8 -nocrypt

Шаг 10. Установка прав на файлыchmod 600 ca.key server.key client.key client.pk8
```

Примечание

Если клиентский ключ планируется использовать от лица текущего пользователя, назначьте права 600 (u=rw), если от root — 640 (u=rw,g=r).

Если требуется использовать хранилище в формате PKCS#12, то следует:

- объединить сертификат и ключ клиента в один файл (`cat client.key client.crt > store.txt`)
- конвертировать получившийся файл в формат PKCS#12 (`openssl pkcs12 -export -in store.txt -out store.pkcs12 -name myAlias -noiter -nomaciter`)

Настройки для mTLS-коммуникации внутри кластера

Каждому экземпляру Picodata нужен сертификат для подключения к другим членам кластера под учетной записью `pico_service`.

Допустим наш кластер состоит из трех серверов:

- `server1.int`
- `server2.int`
- `server3.int`

Эти же имена мы указываем в конфигурациях инстансов Picodata для параметра `instance.ipproto.advertise`.

Сгенерируйте CA-сертификат как описано выше. Далее создайте 3 сертификата с указанием учетной записи `pico_service` и корректного имени каждого сервера.

```
# server1.int
openssl genrsa -out server1.int.key 2048
openssl req -new -key server1.int.key -out server1.int.csr -subj "/CN=pico_service@server1.int"
openssl x509 -req -in server1.int.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server1.int.crt -days 3650 -sha256 < (echo subjectAltName="DNS:server1.int"
)

# server2.int
openssl genrsa -out server2.int.key 2048
openssl req -new -key server2.int.key -out server2.int.csr -subj "/CN=pico_service@server2.int"
openssl x509 -req -in server2.int.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server2.int.crt -days 3650 -sha256 < (echo subjectAltName="DNS:server2.int"
)

# server3.int
openssl genrsa -out server3.int.key 2048
openssl req -new -key server3.int.key -out server3.int.csr -subj "/CN=pico_service@server3.int"
openssl x509 -req -in server3.int.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server3.int.crt -days 3650 -sha256 < (echo subjectAltName="DNS:server3.int"
)
```

Примечание

Если в `instance.ipproto.advertise` указывается IP-адрес сервера, а не DNS-имя, то при создании сертификата нужно указывать IP-адрес сервера в параметре `subjectAltName`. Например, `subjectAltName="IP:192.168.1.1"`.

Вместо этого также можно выпустить один шаблонный сертификат, который будет годиться для любого сервера в домене `.int`:

```
# *.int
openssl genrsa -out wildcard.int.key 2048
openssl req -new -key wildcard.int.key -out wildcard.int.csr -subj "/CN=pico_service"
openssl x509 -req -in wildcard.int.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out wildcard.int.crt -days 3650 -sha256 < (echo subjectAltName="DNS:*.int")
```

Настройка mTLS для коммуникации внутри кластера

Для защиты коммуникации внутри кластера по протоколу IPROTO используйте безопасный режим mutual TLS (mTLS). Он работает следующим образом:

- Администратор создает корневой сертификат (CA), набор серверных и клиентских сертификатов и ключей и подписывает их с помощью CA (см. выше).
- Администратор настраивает поля сертификата `server.crt` (`instance.ipproto.tls.cert_file`):
 - При взаимодействии инстансов друг с другом `server.crt` выступает одновременно как серверный, и клиентский сертификат. Поэтому, при добавлении в сертификат поля `Extended Key Usage`, в нем должны быть оба значения: и `TLS Web Client Authentication`, и `TLS Web Server Authentication`.
 - Поле `subjectAltName` сертификата `instance.ipproto.tls.cert_file` должно соответствовать строке, указанной в параметре `instance.ipproto.advertise`. Например, если в настройке инстанса указано: `ipproto: advertise: myserver1.int:13001` то в поле `subjectAltName` сертификата должно быть значение `myserver1.int` или `wildcard *.int`.
 - Поле `CN` должно содержать значение `pico_service` — служебного пользователя для авторизации запросов внутри кластера. Допустимы значения `CN=pico_service` или в виде `CN=pico_service@myserver1.int`.
- На стороне сервера (инстанса Picodata) администратор включает режим TLS и предоставляет пути к CA, а также серверному сертификату и закрытому ключу. CA

должен быть единым для всех экземпляров кластера; содержимое серверных сертификатов и закрытого ключа должно быть идентичным на каждом экземпляре. Пример настройки mTLS в файле конфигурации:

```
ipproto:
  tls:
    enabled: true
    cert_file: tls/server.crt
    key_file: tls/server.key
    ca_file: tls/ca.crt
```

Примечание

mTLS должен быть включен на всех экземплярах кластера

- После запуска экземпляров кластера внутренняя коммуникация по протоколу IPROTO будет происходить в зашифрованном виде и в режиме проверки TLS verify-full. Администратор может подключиться к любому экземпляру кластера, предоставив CA, клиентский сертификат и клиентский закрытый ключ. Если указан набор флагов --tls-auth, --tls-cert --tls-key и --tls-ca, то при успешной проверке сертификатов и ключа на стороне сервера (экземпляра), администратор получит доступ к кластеру без пароля. Подключение возможно в командной строке с помощью команд picodata status, picodata expel и picodata plugin configure.

Настройка HTTPS для безопасного доступа к метрикам и веб-интерфейсу

Для безопасного доступа к метрикам и веб-интерфейсу Picodata настройте шифрование протокола HTTP. Для этого следует указать в конфигурационных файлах экземпляров соответствующий блок настроек:

```
http:
  tls:
    enabled: true
    cert_file: cert.pem
    key_file: key.pem
```

Для генерации необходимых сертификатов и ключей используйте приведенные выше инструкции.

Использование журнала аудита

Общие сведения

На каждом экземпляре Picodata можно включить регистрацию событий и запись соответствующей информации в журнал аудита.

Журнал аудита позволяет администратору отслеживать все важные события, которые влияют на безопасность кластера Picodata, включая, например, создание новых учетных записей, изменение их паролей и привилегий, и т.д.

Способы ведения журнала

Каждый узел кластера ведет свой собственный журнал, но настройка параметров журнала производится централизованно. В зависимости от настройки, журнал может вестись следующими способами:

- `file` — в виде текстового файла, по строчке на событие. В таком случае смотреть его можно средствами защищенной ОС, а также при помощи API, предоставляемых самой СУБД.
- `pipe` — с использованием внешнего процесса-коллектора, который СУБД должна будет запустить при старте узла. В этом случае журнал будет поступать в текстовом виде на вход процессу-коллектору (через `stdin`). Вся дальнейшая логика работы целиком зависит от запущенного процесса; например, он может отправлять журнал в централизованное внешнее хранилище.
- `syslog` — с использованием общесистемного журналирования и встроенных средств защищенной ОС. Подобная схема решает вопросы архивации, ротации и оповещения администратора. В качестве реализации Syslog на современных дистрибутивах Linux часто используется `journald` (компонент системного менеджера `Systemd`), который имеет множество настроек и даже позволяет отправлять журналы на внешние хосты.

По умолчанию, запись событий не ведется.

Способ вывода журнала задается при запуске экземпляра. После первоначального бутстрапа экземпляра эту настройку можно изменить в дальнейшем, перезапустив экземпляр с новым значением параметра.

Включение журнала

Задайте способ ведения журнала при запуске экземпляра. Набор действий зависит от того, каким способом вы запускаете экземпляры кластера.

В командной строке

Используйте параметр `picodata run --audit` в командной строке:

Вывод журнала в текстовый файл:

```
picodata run --audit=/tmp/audit.log
```

Вывод журнала в syslog:

```
picodata run --audit=syslog:
```

В файле конфигурации

Используйте параметр `instance.audit` в файле конфигурации:

1. Сгенерируйте шаблон файла конфигурации со значениями по умолчанию: `picodata config default -o config.yaml`
2. Отредактируйте полученный файл `config.yaml`, задав нужные значения у параметров. В том числе, измените строку `audit: null`, указав требуемый способ вывода журнала.

Вывод журнала в текстовый файл: `audit: file:/tmp/audit.log` Вывод журнала в syslog: `audit: syslog:`

3. Запустите инстанс, используя параметры из файла конфигурации: `picodata run --config config.yaml`

С помощью переменной окружения

Задайте нужное значение для переменной `PICODATA_AUDIT_LOG`.

Вывод журнала в текстовый файл:

```
PICODATA_AUDIT_LOG=audit.log picodata run
```

Вывод журнала в syslog:

```
PICODATA_AUDIT_LOG=syslog: picodata run
```

Необходимый набор переменных иногда удобнее задать в shell-скрипте. Например:

```
#!/bin/bash
export PICODATA_AUDIT_LOG=syslog:
picodata run
```

С помощью Ansible

Используйте переменные `audit`, `audit_to` и `audit_pipe_command` при настройке роли Picodata для Ansible:

- `audit` — включение аудита событий в кластере. Может принимать значения `true` и `false`
- `audit_to` — способ ведения журнала аудита. Может принимать значения `syslog` (по умолчанию), `file` и `pipe`
- `audit_pipe_command` — команда для перенаправления сообщений аудит в подпроцесс в том случае, если в `audit_to` задано значение `pipe`

Порядок действий для использования переменных:

1. Ознакомьтесь с инструкцией по Развертыванию кластера с помощью Ansible
2. Создайте инвентарный файл `hosts/cluster.yaml` и добавьте в него нужный набор параметров аудита. Вывод журнала в текстовый файл (файлы аудита будут размещаться в `log_dir` и начинаться с `audit-`): Вывод журнала в syslog:

```
all:
vars:
  log_dir: /var/log/picodata
  audit: true
  audit_to: file
```

```
all:
vars:
  audit: true
  audit_to: syslog
```

3. Создайте плейбук `playbooks/picodata.yml` (см. подробнее)

4. Установите кластер:

```
ansible-playbook -i hosts/cluster.yml playbooks/picodata.yml
```

Получение доступа к журналу

Если журнал выводится в отдельный текстовый файл, то отслеживать изменения в нем можно следующей командой:

```
tail -f /tmp/audit.log
```

Вывод журнала в `syslog` в современных дистрибутивах Linux предполагает доступ к данным с помощью `journalctl` (компонента `Systemd`). Для изоляции событий, относящихся только к экземпляру `Picodata`, используйте такую команду:

```
journalctl -f | grep tarantool
```

Если запись в `syslog` происходит через скрипт-обработчик (пример), то команда для доступа к журналу будет немного отличаться:

```
journalctl -f | grep AUDIT
```

Включение журнала для DML-операций

По умолчанию, DML-операции (`INSERT/UPDATE/DELETE`) не записываются в журнал аудита. Эту функциональность можно включить через SQL-запрос для конкретных пользователей:

- `AUDIT POLICY dml_default BY dbuser` — включить регистрацию DML-операций для пользователя `dbuser`
- `AUDIT POLICY dml_default EXCEPT dbuser` — отключить регистрацию DML-операций для пользователя `dbuser`

Примечание

Включение журнала для DML-операций влияет на производительность системы (на число выполняемых запросов в единицу времени). Согласно данным бенчмарка `pgbench`, снижение составляет ~5%.

Оповещения о событиях безопасности

Поддерживаемые способы доставки оповещений

В `Picodata` существует возможность получать оповещения о событиях, зафиксированных в журнале аудита. Поддерживаются следующие способы доставки оповещений:

- уведомления для графического рабочего стола
- уведомления по электронной почте

Оповещения для рабочего стола

Для доставки оповещений на рабочий стол используется программа `notify-send`, имеющаяся в составе защищенной ОС. Для того, чтобы использовать ее совместно с `Picodata`, потребуется создать файл-обработчик следующего содержания:

```
#!/usr/bin/env python

import json
import subprocess
import sys
from systemd import journal

for line in sys.stdin:
    if line:
        journal.send(f"AUDIT {line}")
        entry = json.loads(line)
        if entry['severity']:
            subprocess.call(['notify-send', entry['message']])
```

Сохраните этот код в файл `audit-sink.py` в директорию, из которой будет запускаться `picodata`, сделайте его исполняемым (`chmod +x audit-sink.py`) и затем запустите инстанс:

`picodata run --admin-sock ./il.sock --audit='|/tmp/audit-sink.py'` В результате, сообщения из журнала аудита будут сохраняться в `syslog` и дублироваться в виде уведомлений на рабочем столе.

Оповещения по электронной почте

Для получения оповещений по электронной почте потребуются:

- программа `sendmail`, имеющаяся в составе защищенной ОС
- настройка `sendmail` для корректной работы с почтовым сервером
- файл-обработчик для дублирования событий журнала аудит в виде сообщений для `sendmail`

Для настройки `sendmail` нужно отредактировать файл настроек `/etc/mail/sendmail.mc` следующим образом:

```
define(`SMART_HOST', `your.smtp.server')dnl
define(`confAUTH_MECHANISMS', `EXTERNAL GSSAPI DIGEST-MD5 CRAM-MD5 LOGIN PLAIN')dnl
FEATURE(`authinfo', `hash -o /etc/mail/authinfo.db')dnl
```

где `your.smtp.server` — адрес SMTP-сервера для отправки писем

Далее следует настроить аутентификацию, отредактировав файл `/etc/mail/authinfo`:

```
AuthInfo:your.smtp.server "U:your_username" "P:your_password" "M:PLAIN"
```

где:

- `your.smtp.server` — адрес SMTP-сервера для отправки писем
- `your_username` — имя пользователя
- `your_password` — пароль пользователя
- `PLAIN` — тип аутентификации

Следующим шагом нужно сгенерировать базу данных для аутентификации:

```
sudo makemap hash /etc/mail/authinfo < /etc/mail/authinfo
```

сгенерировать конфигурацию sendmail:

```
sudo make -C /etc/mail
```

включить и запустить sendmail:

```
sudo systemctl enable --now sendmail.service
```

Теперь можно использовать модифицированный скрипт `audit-sink.py` со следующим содержанием:

```
#!/usr/bin/env python

import json
import subprocess
import sys
from systemd import journal

for line in sys.stdin:
    if line:
        journal.send(f"AUDIT {line}")
        entry = json.loads(line)
        if entry['severity']:
            subprocess.call(['sendmail user@example.com <', entry['message']])
```

В результате, сообщения из журнала аудита будут попадать в `syslog` от имени процесса `python` и дублироваться в виде писем на указанный вместо `user@example.com` почтовый ящик.

Рекомендации по сайзингу

В данном разделе приведены рекомендации по сайзингу вычислительных ресурсов при планировании нагрузки на кластер Picodata. Эта информация поможет вам рассчитать необходимые вычислительные ресурсы для использования СУБД Picodata.

Задачи сайзинга

Сайзинг помогает подобрать вычислительные ресурсы (количество ядер процессора, объем оперативной памяти, дисковое пространство и т.д.) в соответствии с возможностями базы данных Picodata.

Сайзинг заключается в:

- оптимальном распределении уже имеющихся ресурсов для запуска кластера Picodata.
- расчете необходимого количества аппаратных ресурсов для Picodata исходя из оценочного размера хранимых данных.

Распределение имеющихся ресурсов

Для одного узла Picodata необходимо:

- **CPU** — при использовании движка memtx следует выделять на каждый инстанс Picodata 1 ядро CPU. При использовании движка vinyl — 3 ядра (из-за интенсивного дискового I/O)
- **RAM** — объем выделяемой памяти на инстанс зависит от объема данных, который будет на нем храниться (см. ниже), однако рекомендуется выделять от 8 ГБ (для движка memtx). При распределении значительного пула ресурсов RAM рекомендуется выдавать каждому узлу Picodata 32 ГБ RAM.
- **дисковое хранение** — для движка memtx оптимальный размер дискового хранения рассчитывается как трехкратный размер RAM. Для движка vinyl рекомендуемое значение можно получить только после детального планирования состава и динамики роста БД.

Для определения количества узлов следует разделить общий размер БД на размер 1 узла (например, 32 ГБ) и умножить результат на число реплик.

Вычисление нагрузки

Расчет ожидаемого размера БД

Размер БД можно оценить следующим образом:

1. Выяснить размер полей, исходя из того, какие данные планируется хранить
2. Выяснить размер кортежа (строки в базе данных), сложив размеры полей
3. Получить размер таблицы, умножив размер кортежа на количество строк
4. Сложить размеры всех таблиц
5. Прибавить размеры индексов

Расчет размеров полей

Размер значений в таблице зависит от их типа, из-за чего в *некоторых* случаях используется дополнительный префиксный байт.

Примеры размеров значений разных типов:

Целые числа (INTEGER):

- 0-127 — 1 байт (без префикса)
- 127-255 — 2 байта (без префикса)
- 255-65535 — 3 байта (без префикса)
- 65536 — 5 байт (1 префиксный байт + 4 байта uint32)
- 10000000000 — 9 байт (1 префиксный байт + 8 байт uint64)

UUID-идентификаторы (UUID):

- 102be9bf-215f-4180-b4c9-cc11983e988d — 18 байт (2 префиксных байта массива + 16 байт для всего значения UUID)

Текст (TEXT):

- abc — 4 байта (1 префиксный байт + 3 байта на текстовые символы)
- John Doe — 9 байт (1 префиксный байт + 8 байт на текстовые символы)

Дата/время (DATETIME):

- 2024-10-18 23:59:59 — 10 байт
- 2024-10-18 — 10 байт (отсутствующие значения будут заполнены нулями, например 2024-10-18 00:00:00)

Пустое значение поля (nil) в непустом кортеже занимает 1 байт.

Расчет размера кортежей

Размер кортежа представляет собой сумму размеров полей, входящих в его состав, плюс 4 байта смещения, требуемое форматом msgrack для всех значений кроме 0/nil.

Проиллюстрируем это примерами таблиц с разным числом колонок:

Размер: 5 байт+----+

```
| id |
+====+
| 1 |
+----+
```

Размер: 9 байт+----+-----+

```
| id | name |
+=====+
| 1 | bob |
+----+-----+
```

Размер: 27 байт+----+-----+-----+-----+-----+

```
| id | name | uuid |
+-----+-----+-----+-----+-----+
| 1 | bob | 989259df-7435-429b-86bc-6ecf2c125197 |
+----+-----+-----+-----+-----+
```

Размер: 30 байт+----+-----+-----+-----+-----+

```
| id | name | uuid | year |
+-----+-----+-----+-----+-----+
```

```
| 1 | bob | 989259df-7435-429b-86bc-6ecf2c125197 | 1989 |
+---+-----+-----+-----+-----+
Размер: 18 байт+---+-----+-----+-----+-----+
| id | name | uuid | year | date |
+-----+-----+-----+-----+-----+
| 1 | nil | nil | nil | 2024-10-18T23:59:59Z |
+---+-----+-----+-----+-----+
```

Расчет размера таблицы

Зная размер кортежа, можно рассчитать примерный размер таблицы без индексов. Для этого размер кортежа в байтах следует умножить на количество этих кортежей (т.е. строк в таблице). Для представления величины в Мб нужно полученное значение дважды разделить на 1024. Пример:

```
Размер кортежа: 30 байт
Количество кортежей: 100 000 000
Размер таблицы: 30 * 100 000 000 / 1024 / 1024 = 2861 Мб
```

Расчет индексов БД

Размер отдельного индекса БД можно оценить следующим образом: количество кортежей в таблице умножить на 18 (число байт на индексные данные) и затем умножить на количество индексов в таблице. Если предполагается, к примеру, иметь 2 индекса для таблицы, то общий размер можно вычислить так:

```
Количество кортежей: 100 000 000
Количество индексов: 2
Общий размер индексов в таблице: 100 000 000 * 18 * 2 / 1024 / 1024 = 3434 Мб
```

Расчет требуемого объема для движка memtx

Если в БД будет только одна таблица, то для приведенного выше примера общий объем памяти, необходимый для хранения с использованием движка memtx, составит $2861 + 3434 = 6301$ Мб. Если таблиц будет несколько, то следует посчитать сумму размеров всех таблиц и всех индексов.

Полученный объем относится к **пользовательскому** аллокатору memtx (параметр `instance.memtx.memory`). Для системных таблиц кластера используется отдельный аллокатор с собственной квотой `instance.memtx.system_memory` (по умолчанию 256 МБ); ее также необходимо прибавлять к итоговому объему оперативной памяти, выделяемой на инстанс.

Пример расчета размера таблицы

При расчете размера таблицы используйте в качестве отправной точки размеры кортежа с учетом типа и длины значений в нем. Умножив его на число строк, можно получить примерный размер таблицы.

Реальный размер таблицы в памяти может быть на 5-20% больше (в зависимости от количества строк, наличия/отсутствия пустых полей т.д.). Это связано с необходимостью хранить данные бакетов (`bucket_id`) и особенностью хранения массивов данных в формате msgpack.

Для примера сгенерируем тестовую таблицу с помощью Picostress — генератора нагрузки, развиваемого командой Picodata. Исходным файлом конфигурации для этой таблицы будет готовый шаблон.

Результирующая таблица будет содержать 100 строк следующего вида:

таблица cars_1				
id	mark	rating	uid	dt
0	RiIicFq	0	22301fb8-d829-48da-b007-b05614969f34	1988-01-06T05:14:08+0300
1	RiIicFq	-2	03d00626-73f5-4944-9579-8d340c0a17e8	1991-04-04T22:14:46+0300
2	RiIicF	-2	f9c93b9d-209d-4fc7-8d92-b4a1351f2a09	2010-04-16T05:35:24+0400
...				

При среднем размере кортежа 45-49 байт, фактический размер этой таблицы в памяти составит 5045 байт.

Экстраполяция при подсчете всегда предполагает некоторую погрешность (до 20%).

Настройка Systemd

В случае выбора системы управления процессами Systemd вы можете произвести дополнительные настройки Systemd-службы Picodata, например увеличить лимит на количество открытых файлов, поменять таймаут ожидания запуска службы и т.п. Рекомендуемый способ переопределения подобных настроек — изменение инвентарного файла Picodata для Ansible.

Настройки при этом будут применяться для каждого инстанса кластера.

Настройка лимитов в инвентарном файле

По умолчанию для Systemd-службы Picodata выставлены следующие параметры:

```
LimitNOFILE: 65535      # Increase fd limit for Vinyl
LimitCORE: infinity    # Unlimited coredump filesize
TimeoutStartSec: 86400s # Systemd waits until all xlogs are recovered
TimeoutStopSec: 20s    # Give a reasonable amount of time to close xlogs
```

Вы можете переопределить их в инвентарном файле через словарь `systemd_params`, либо в этом же словаре выставить любые другие параметры, которые используются в Systemd-юнитах.

Пример фрагмента инвентарного файла с определением словаря `systemd_params`:

```
all:
  vars:
    systemd_params:
      LimitNOFILE: 900000
      TimeoutStartSec: '30m'
```

Системные ограничения

Некоторые параметры не могут иметь значение выше установленного в операционной системе, поэтому для их выставления предварительно нужно увеличить системные параметры через `sysctl` или `limits.conf`.

Например, параметр `LimitNOFILE` не может превышать системное значение `fs.nr_open`— если вы хотите выставить большее значение, то предварительно нужно изменить значение в системе:

Узнать текущее значение:

```
sysctl fs.nr_open
```

Отредактируйте файл `/etc/sysctl.conf` (при отсутствии создайте его), выставив в нем увеличенное значение параметра `fs.nr_open`. Например:

```
sudo -i
echo "fs.nr_open=999999" >> /etc/sysctl.conf
```

Примените измененные значения:

```
sudo sysctl -p
```

Проверка лимитов

Лимиты, выставленные для процесса, можно получить командой:

```
cat /proc/<MainPID>/limits
```

Значение MainPID вы можете получить из вывода команды:

```
systemctl show --property=MainPID picodata
```

где `picodata` — имя службы инстанса.

Устранение неполадок

В данном разделе описаны некоторые часто встречающиеся проблемы, возникающие при эксплуатации кластера Picodata, и предложены способы их решения.

Не удастся включить плагин

Пример ошибки

```
picodata> ALTER PLUGIN uroboros 0.2.10 ENABLE;
raft: proposal dropped
```

Причина

Raft не может обработать этот запрос. Например, в данный момент проходят перевыборы лидера кластера.

Решение

Повторить запрос через некоторое небольшое время (1-5 минут). Подробности о состоянии Raft можно посмотреть в консоли администратора Picodata:

```
Состояние Raft\lua
box.space._raft_state:fselect()

Журнал Raft\lua
box.space._raft_log:fselect()

Данные о main_loop, лидере Raft-группы, терме и т.д.\lua
pico.raft_status()
```

Если инстанс кластера работает со сбоями, то более подробную информацию можно получить в его выводе *stdout*.

Пользователь не может авторизоваться

Пример ошибки

```
Enter password for user:
server responded with error: PasswordMismatch: User not found or supplied credentials are invalid
```

Причина

1. Для пользователя не был задан пароль (например, в новом кластере)
2. Для авторизации используется неверный пароль
3. Для авторизации используется неверный метод аутентификации
4. Пользователь был ранее явно заблокирован другим пользователем
5. Пользователь был ранее заблокирован из-за превышения попыток неудачного входа (например, пароль пользователя был изменен, но для авторизации по какой-то причине был несколько раз указан старый пароль)

Решение

1. Если у пользователя нет пароля, то его нужно задать (пример для администратора)
2. Использовать верный пароль

3. Проверить, что для авторизации используется корректный метод аутентификации. Данный совет актуален в том случае, если ранее для пользователя этот метод был переопределен

4. Авторизоваться под другим пользователем с административным доступом (если его нет, то использовать файл `admin.sock`) и разблокировать пользователя:

```
ALTER USER user LOGIN;
```

Невозможно подключиться к инстансу

Пример ошибки

```
Connection refused. Is the server running on that host and accepting TCP/IP connections?
pgbench: error: could not create connection for client 192
```

или

```
{ code: 24, kind: Uncategorized, message: "Too many open files" }
```

Причина

Причины, по которым подключение к инстансу завершается ошибкой, разнообразны. Одна из них связана с тем, что при большом количестве подключений на стороне сервера (инстанса Picodata) может быть исчерпан лимит на файловые дескрипторы.

Решение

Увеличить лимит на файловые дескрипторы.

На уровне системы:

```
sudo sysctl -w fs.file-max=1048576
```

На уровне процесса:

```
sudo sysctl -w fs.nr_open=999999
```

Для персистентного хранения новых значений отредактируйте файл `/etc/sysctl.conf`, вписав в него указанные параметры, и затем примените изменения:

```
sudo sysctl -p
```

В ряде случаев может потребоваться увеличить ограничение, накладываемое на ОС со стороны Systemd. Для этого отредактируйте файл `/etc/systemd/system.conf`, вписав туда нужное значение параметра `DefaultLimitNOFILE`.

Внимание!

Увеличение количества дескрипторов увеличит потребление системных ресурсов

Выпадение инстанса из кластера

Описание

Инстанс в какой-то момент выходит из состава кластера: находится в состоянии `Offline`. Перезапуск инстанса не помогает.

Причина

Инстанс содержит устаревшие данные, не позволяющие ему в текущем виде присоединиться к кластеру

Решение

- остановить проблемный инстанс
- узнать его UUID, например, с помощью команды `picodata status --peer <любой работающий инстанс>`
- исключить проблемный инстанс из кластера: `picodata expel <UUID> --peer <любой работающий инстанс>`
- удалить `snar`- и `xlog`-файлы проблемного инстанса
- запустить проблемный инстанс

Примечание

Если инстанс управляется службой Systemd, то для остановки/запуска следует использовать соответствующие варианты команды `systemctl`

Невозможность добавления новых инстансов в кластер

Описание

В кластере с несколькими тирами и большим числом репликасетов недоступность инстанса, указанного как `instance.peer`, может привести к невозможности добавления новых инстансов в кластер.

Причина

В руководстве Создание кластера приведены примеры файлов конфигурации для кластера из нескольких тиров. Поскольку это все еще небольшой кластер, для всех его инстансов рекомендовано использовать один и тот же `instance.peer` — это первый инстанс кластера. Однако, при масштабировании кластера и разделении его частей на функциональные роли с помощью тиров, такой подход может быть неоптимальным: недоступность `instance.peer` будет влиять на невозможность расширения кластера, а также возникнет проблема при ребутстрапе самого инстанса, указанного в `instance.peer` (он не будет видеть сам себя).

Решение

Для сложных кластеров рекомендуется использовать для параметра `peer` массив инстансов. Например:

```
peer:
- 192.168.101.167:3301
- 192.168.101.168:3301
- 192.168.101.166:3301
```

Желательно, чтобы эти инстансы были расположены на разных серверах кластера.

Таким образом, недоступность отдельных узлов, указанных в списке, не повлияет на связность инстансов в кластере.

Сообщение об ошибке повторного подключения к репликасету

Описание

Когда инстанс перезапускается и заново подключается к мастер-реплике своего репликасета, в журнале сообщений инстанса можно увидеть сообщение об ошибке следующего вида:

```
localhost [3369449] main/392/applier/pico_service@192.169.1.7:3190 box.cc:4483 E> ER_CFG: Incorrect value for option 'replication': duplicate connection with the same replica UUID
```

Причина

Сообщение появляется из-за того, что инстанс пытается переподключиться к репликасету до того, как его мастер-реплика обработает предыдущую попытку подключения.

Решение

Сообщение о дубликате подключения не является ошибкой и может быть проигнорировано.

Информационная безопасность

Действия по приемке поставленного ПО

Приемка поставленного ПО состоит из следующих действий:

- проверка требований к сырью, материалам и комплектующим;
- проверка комплектности;
- проверка комплектности и качества эксплуатационной документации;
- проверка маркировки и упаковки;
- проверка соответствия контрольных сумм.

Проверка требований к сырью, материалам и комплектующим

Проверка проводится только для экземпляров изделия, поставляемых на материальных носителях. Проверка проводится путем внешнего осмотра CD/DVD-диска, USB-флеш-накопителя и упаковки. Проверка считается успешной, если:

- для изготовления использовались диски типа CD-R/DVD-R с покрытием типа Printable;
- при внешнем осмотре USB-флеш-накопителя и диска на изделие не обнаружено царапин, деформаций, расслоений и загрязнений на их рабочих поверхностях.

Проверка комплектности

При проверке комплектности состав изделия должен соответствовать составу, представленному в документе «Picodata. Формуляр» 40429836.01.62.0 ТД.ФО.

Проверка комплектности и качества эксплуатационной документации

Проверка проводится путем сверки комплектности эксплуатационной документации, а также путем открытия и просмотра на рабочей станции эксплуатационной документации, представленной на компакт-диске.

Проверка считается успешной, если:

- комплектность эксплуатационной документации соответствует составу, приведенному в документе «Picodata. Формуляр» 40429836.01.62.0 ТД.ФО;
- эксплуатационная документация, представленная на компакт-диске, открывается на рабочей станции и пригодна для чтения с экрана монитора и вывода на бумажный носитель.

Проверка маркировки и упаковки

Проверка проводится путем наличия маркировки.

Проверка считается успешной, если формуляр изделия содержит идентификатор Системы сертификации средств защиты информации по требованиям безопасности

информации ФСТЭК России.

Требования к упаковке не предъявляются.

Действия по проверке контрольных сумм

Проверка контрольных производится посредством сравнения контрольных сумм дистрибутива и исполняемых файлов, указанных в документе «Picodata. Формуляр» 40429836.01.62.0 ТД.ФО и результатом снятия контрольных сумм ответственным специалистом пользователя изделия.

Расчет контрольных сумм осуществляется посредством использования программы «gostsum v2.2.0» по алгоритму ГОСТ Р 34.11-2012 (gost-2012) из состава сертифицированной операционной системы Astra Linux Special Edition (сертификат ФСТЭК России № 2557, выдан 27 января 2012 года, действителен до 27 января 2031 года).

Действия по безопасной установке и настройке изделия

Для безопасной установки и настройки изделия аппаратная и программная часть среды функционирования должна удовлетворять минимальным системным требованиям, указанным в документе «Picodata. Формуляр» 40429836.01.62.0 ТД.ФО.

Действия по реализации функций безопасности среды функционирования изделия

Работа изделия должна обеспечиваться путем развертывания изделия в сертифицированной операционной системе:

- ОС Альт 8 СП релиз 9 (сертификат соответствия ФСТЭК России № 3866);
- ОС Альт 8 СП релиз 10 (сертификат соответствия ФСТЭК России № 3866);
- ОС «РЕД ОС» версии 7 (сертификат соответствия ФСТЭК России №4060);
- ОС «РЕД ОС» версии 8 (сертификат соответствия ФСТЭК России № 4060);
- ОС «Astra Linux Special Edition» версии 1.7 (сертификат соответствия ФСТЭК России № 2557);
- ОС «Astra Linux Special Edition» версии 1.8 (сертификат соответствия ФСТЭК России № 2557);
- ОС РОСА «ХРОМ» платформа 2021.1 (сертификат соответствия ФСТЭК России №4818).

Настройка операционной системы должна быть выполнена в соответствии с эксплуатационной документацией на операционную систему.